

DOSSIER PROJET

Cursus Ingénieur DevOps

RNCP36061

Projet Starling

Plateforme de visualisation et d'analyse
des données alimentaires mondiales

Hugo PRADIER

Promotion Novembre 2024

DataScientest





Remerciements

Je tiens à exprimer ma profonde gratitude envers tous ceux qui ont contribué à la réussite de ce projet fil rouge et à l'accomplissement de ma formation Ingénieur DevOps chez DataScientest.

Mes remerciements s'adressent en premier lieu à **Benjamin Fiche**, mon mentor projet, dont l'accompagnement exceptionnel, l'expertise technique et le soutien sans faille ont été déterminants dans la réussite de ce projet. Sa disponibilité, ses conseils avisés et sa capacité à me guider à travers les défis techniques les plus complexes ont été d'une valeur inestimable, particulièrement dans les moments difficiles du projet.

Je remercie également l'équipe pédagogique de DataScientest pour la qualité de la formation bootcamp et leur accompagnement tout au long du cursus. Leur expertise m'a permis d'acquérir les compétences techniques nécessaires à la réalisation de ce projet DevOps d'envergure.

Ma gratitude va aussi à mon chef de cohorte pour son suivi régulier et ses encouragements, notamment lors de la période où j'ai dû poursuivre le projet seul après le départ de mes partenaires de formation.

Enfin, je tiens à souligner l'importance de la communauté DevOps open-source dont les ressources documentaires, forums d'entraide et outils ont été essentiels pour surmonter les défis techniques de ce projet.





Table des matières

Remerciements	2
Introduction.....	6
Accroche	6
Contexte et enjeux	6
Définition des termes clés	6
Présentation du projet Starling.....	8
Environnement technique et architecture	9
Annonce du plan	10
1. Compétences du référentiel RNCP36061 couvertes par le projet.....	11
1.1. Bloc de compétences 1 : Automatiser le déploiement d'une infrastructure dans le cloud	11
1.1.1. Automatiser la création de serveurs à l'aide de scripts	11
1.1.2. Automatiser le déploiement d'une infrastructure.....	11
Diagramme Gestion du Remote State Terraform avec S3 et DynamoDB	12
1.1.3. Sécuriser l'infrastructure	13
1.1.4. Mettre l'infrastructure en production dans le cloud	13
1.2. Bloc de compétences 2 : Déployer en continu une application	14
1.2.1. Préparer un environnement de test.....	14
1.2.2. Gérer le stockage des données	14
1.2.3. Gérer des containers	14
1.2.4. Automatiser la mise en production d'une application avec une plateforme	15
.....	18
1.3. Bloc de compétences 3 : Superviser les services déployés.....	19
1.3.1. Architecture de supervision déployée	19
Stack Prometheus + Grafana sur EKS	19
Centralisation des logs avec CloudWatch	19
Monitoring des files SQS.....	20
1.3.2. Définir et mettre en place des statistiques de services	21
Métriques infrastructure (niveau cluster).....	21
Métriques nodes (niveau infrastructure)	22
Métriques applicatives (niveau pods)	23
Métriques réseau (niveau cluster)	24
1.3.3. Exploiter une solution de supervision	25
Dashboards Grafana déployés	25
Utilisation opérationnelle	25
Alerting (perspectives).....	25
1.3.3. Échanger sur des réseaux professionnels éventuellement en anglais	27
1.4. Synthèse de la couverture des compétences RNCP36061	28
2. Cahier des charges du projet Starling	29
2.1. Contexte et genèse du projet	29
2.2. Objectifs du projet.....	29



2.3. Contraintes du projet.....	29
2.4. Livrables attendus	30
3. Description de la démarche et des outils utilisés	32
3.1. Organisation du projet et évolution de l'équipe	32
3.1.1. Composition initiale et répartition des tâches	32
3.1.2. Difficultés organisationnelles et réorganisation	32
3.2. Démarche technique : de la version EC2 à la version EKS	33
3.2.1. Phase 1 : Version initiale sur EC2 (avec l'équipe)	33
3.2.2. Phase 2 : Reconstruction complète sur Amazon EKS (solo)	34
3.3. Outils de collaboration et méthodes de travail	35
3.3.1. Phase collaborative (équipe de 3 puis 2 personnes)	35
3.3.2. Phase solo avec accompagnement mentor	36
3.3.3. Veille technologique et communauté DevOps	36
3.4. Synthèse de la démarche.....	36
4. Réalisations du candidat.....	37
4.1. Infrastructure as Code : Configuration Terraform du cluster EKS	37
4.1.1. Présentation et objectif	37
4.1.2. Choix techniques majeurs	37
4.1.3. Architecture réseau sécurisée (VPC)	38
4.1.4. Cluster EKS avec add-ons essentiels	38
4.1.5. Karpenter pour l'autoscaling intelligent	39
4.1.6. Intégration AWS Load Balancer Controller	40
4.1.7. Gestion des identités avec EKS Pod Identity	40
4.2. Pipeline CI/CD : Automatisation GitLab CI.....	41
4.2.1. Présentation et objectif	41
4.2.2. Architecture du pipeline : 5 stages	41
4.2.3. Stage 1 : Scan de sécurité (Secret Detection).....	42
4.2.4. Stage 2 : Build des images Docker (Backend & Frontend).....	42
4.2.5. Stages 3, 4, 5 : Déploiements progressifs (Dev, Preprod, Prod)	44
5. DESCRIPTION D'UNE SITUATION DE TRAVAIL AYANT NÉCESSITÉ UNE RECHERCHE.....	46
5.1. Contexte de la problématique.....	46
5.2. Démarche de recherche	46
5.2.1. Sources consultées initialement	46
5.2.2. Découverte de la solution : AWS EKS Blueprints	47
5.3. Implémentation de la solution	47
5.3.1. Exploration du repository EKS Blueprints	47
5.3.2. Adaptation au projet Starling	48
6. CONCLUSION ET OUVERTURE.....	49
6.1. Synthèse du projet et réponse aux problématiques	49
6.2. Difficultés rencontrées et solutions apportées	49
Défi organisationnel : Passage d'un projet d'équipe à un projet solo	50
Contrainte budgétaire : Déployer une infrastructure production-ready avec 400€.....	50
Complexité technique : Courbe d'apprentissage Kubernetes et EKS	50
Contraintes personnelles : Problèmes de santé impactant la disponibilité	50



DOSSIER PROJET

6.3. Principaux bénéfices tirés du projet	50
6.4. Préconisations pour l'évolution du projet	51
Court terme (0-3 mois) : Fiabilisation et qualité	52
Moyen terme (3-6 mois) : Scalabilité et performance	52
Long terme (6-12 mois) : Architecture d'entreprise et résilience	52
6.5. Ouverture : Du projet pédagogique à l'impact professionnel	53
Retour sur l'accroche : La technologie au service des enjeux mondiaux	53
Perspectives professionnelles : Devenir Data Platform Engineer dans la santé	54
Engagement dans l'amélioration continue	54
ANNEXES	55
Annexe A : Ressources documentaires officielles	55
Documentation AWS	55
Documentation Kubernetes et CNCF	55
Documentation Infrastructure as Code	56
Documentation GitOps et CI/CD	56
Documentation Monitoring et Observabilité	57
Documentation Frameworks Applicatifs	57
Annexe B : Ressources AWS EKS Blueprints	58
Annexe C : Blogs techniques et veille technologique	58
Annexe D : Communautés et forums	59
Annexe E : Outils et utilitaires DevOps	59
Annexe F : Certifications et formations	60
Annexe G : Organismes et institutions du secteur alimentaire	61
Annexe I : Ressources complémentaires	61



Introduction

Accroche

« Selon la FAO, près de 828 millions de personnes souffrent encore de la faim dans le monde en 2024. » Face à cette réalité alarmante, la transparence et l'accessibilité des données alimentaires mondiales deviennent des enjeux cruciaux pour comprendre et résoudre les problématiques de sécurité alimentaire à l'échelle globale. Dans ce contexte, la capacité à collecter, traiter et visualiser efficacement ces données constitue un levier essentiel pour les décideurs politiques, les chercheurs et les organisations internationales.

Contexte et enjeux

La sécurité alimentaire mondiale représente un défi majeur du 21ème siècle. Face à l'augmentation de la population mondiale, aux changements climatiques et aux inégalités croissantes de distribution des ressources, il devient impératif de disposer d'outils performants permettant d'analyser et de comprendre les dynamiques complexes du système alimentaire mondial.

Les données alimentaires mondiales, bien que disponibles via différentes sources institutionnelles (FAO, Banque mondiale, organisations internationales), restent souvent fragmentées, difficiles d'accès et complexes à interpréter pour les non-spécialistes. Cette dispersion des informations limite leur exploitation par les acteurs du terrain et ralentit la prise de décision en matière de politiques publiques.

Dans ce contexte, le besoin d'une plateforme centralisée, performante et accessible pour visualiser et analyser ces données devient évident. C'est précisément l'objectif du projet Starling.

Définition des termes clés



DOSSIER PROJET

Amazon RDS (Elastic Block Store) Service de stockage par blocs d'AWS pour des volumes persistants attachés aux instances EC2. Utilisé pour la base de données PostgreSQL dans Starling.

Amazon EKS (Elastic Kubernetes Service) Service managé par AWS pour déployer et gérer des applications conteneurisées avec Kubernetes. Orchestrateur principal de Starling.

Amazon Route 53 Service DNS d'AWS pour gérer les noms de domaine et router le trafic utilisateur. Utilisé pour les sous-domaines des environnements Starling.

API (Application Programming Interface) Interface permettant à différentes applications de communiquer. FastAPI expose une API REST pour le frontend Next.js dans Starling.

ArgoCD Outil de déploiement continu pour Kubernetes suivant les principes GitOps. Synchronise l'état du cluster Kubernetes avec les manifestes stockés dans Git.

AWS (Amazon Web Services) Plateforme de services cloud computing. Fournisseur cloud principal de Starling.

CI/CD (Continuous Integration / Continuous Deployment) Pratique DevOps combinant l'intégration continue et le déploiement continu. GitLab CI orchestre ce pipeline dans Starling.

Container (Conteneur) Unité logicielle standardisée qui empaquette le code d'une application avec ses dépendances. Starling utilise Docker pour créer des containers.

DevOps Culture et ensemble de pratiques combinant le développement logiciel et les opérations informatiques. Philosophie centrale du projet Starling.

Docker Plateforme open-source pour créer, déployer et exécuter des applications dans des containers. Utilisé pour conteneuriser les composants de Starling.

ECR (Elastic Container Registry) Service de registre de containers Docker managé par AWS. Utilisé pour stocker les images Docker de Starling.

FastAPI Framework Python pour construire des API REST avec validation automatique des données. Framework backend du projet Starling.

GitLab CI Outil d'intégration et de déploiement continu intégré à GitLab. Orchestrateur principal du CI/CD dans Starling.

GitOps Paradigme opérationnel utilisant Git comme source unique de vérité pour l'infrastructure et les applications déclaratives. Utilisé dans Starling.

Grafana Plateforme open-source d'analyse et de visualisation de métriques. Utilisée pour visualiser les métriques collectées par Prometheus dans Starling.



DOSSIER PROJET

Helm Gestionnaire de paquets pour Kubernetes permettant de définir, installer et mettre à jour des applications Kubernetes complexes. Utilisé pour packager et déployer l'application Starling.

IAM (Identity and Access Management) Service AWS de gestion des identités et des accès. Utilisé pour appliquer le principe du moindre privilège dans Starling.

laC (Infrastructure as Code) Pratique de gestion de l'infrastructure informatique via du code. Terraform implémente l'laC pour l'infrastructure AWS de Starling.

Kubernetes (K8s) Système open-source d'orchestration de containers. Plateforme d'orchestration centrale du projet Starling via Amazon EKS.

Microservices Architecture logicielle structurant une application comme un ensemble de services indépendamment déployables. Starling adopte cette architecture.

Monitoring (Supervision) Processus de collecte et d'analyse de données pour surveiller la performance et la disponibilité des systèmes. Réalisé dans Starling via Prometheus et Grafana.

Next.js Framework React pour créer des applications web performantes. Framework frontend du projet Starling.

PostgreSQL Système de gestion de base de données relationnelle open-source. Base de données principale du projet Starling.

Prometheus Système open-source de monitoring et d'alerting. Collecte les métriques infrastructure et applicatives de Starling.

Registry (Registre Docker) Système de stockage et de distribution d'images Docker. Starling utilise AWS ECR comme registre privé.

Secret (Kubernetes Secret) Objet Kubernetes pour stocker des informations sensibles. Utilisé pour gérer les credentials de base de données dans Starling.

Security Group Pare-feu virtuel AWS contrôlant le trafic entrant et sortant des ressources. Configurés dans Starling selon le principe du moindre privilège.

Terraform Outil open-source d'Infrastructure as Code. Outil principal d'automatisation de l'infrastructure AWS de Starling.

VPC (Virtual Private Cloud) Réseau virtuel isolé dans le cloud AWS. Chaque environnement Starling dispose de son propre VPC pour l'isolation.

Présentation du projet Starling



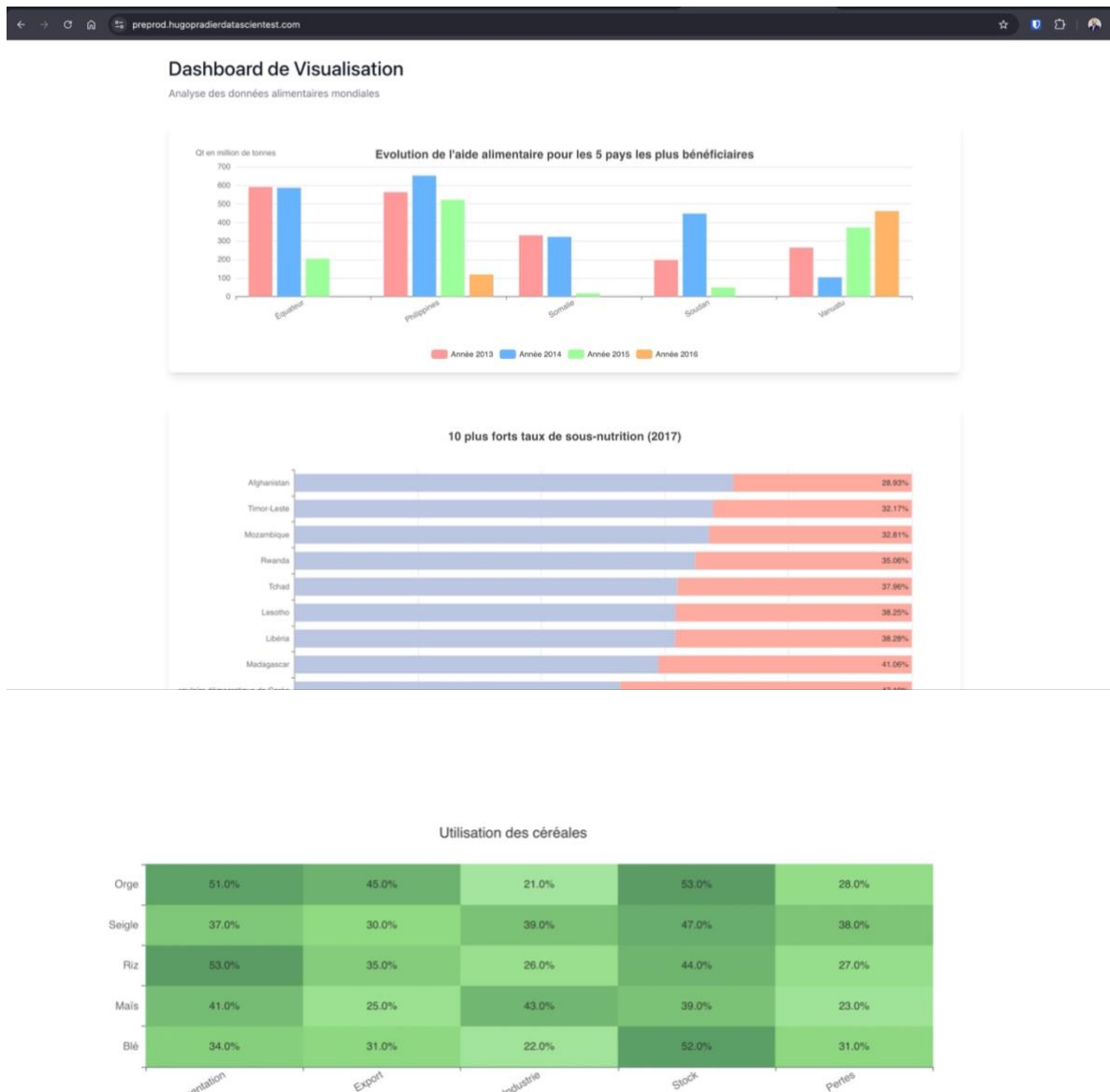
DOSSIER PROJET

Starling est une plateforme web innovante de visualisation et d'analyse des données alimentaires mondiales, développée dans le cadre de ma formation Ingénieur DevOps (RNCP36061) chez DataScientest, promotion Novembre 2024. Ce projet s'inscrit dans une démarche de démocratisation de l'accès aux données sur la sécurité alimentaire mondiale.

L'application permet de visualiser et d'analyser trois indicateurs clés de la sécurité alimentaire :

- L'aide alimentaire internationale et sa distribution géographique
- Le taux de sous-nutrition par pays et par région
- L'utilisation des céréales et les flux commerciaux mondiaux

Voici une capture d'écran de l'application Starling.



Environnement technique et architecture



DOSSIER PROJET

Le projet Starling repose sur une architecture moderne orientée microservices, déployée sur Amazon Web Services (AWS). L'infrastructure s'articule autour des technologies suivantes :

- Frontend : Next.js pour une interface utilisateur réactive et performante
- Backend : FastAPI (Python) pour une API REST rapide et documentée
- Base de données : PostgreSQL hébergée sur AWS RDS
- Orchestration : Kubernetes via Amazon EKS / KARPENTER
- Infrastructure as Code : Terraform
- GitOps : ArgoCD
- Package manager : Helm
- CI/CD : GitLab CI
- Monitoring : Prometheus et Grafana, CloudWatch
- DNS : Amazon Route 53
- Registre Docker: AWS ECR

Annnonce du plan

Ce dossier projet s'articule autour de plusieurs axes structurants. Dans un premier temps, nous présenterons les compétences du référentiel RNCP36061 mobilisées et démontrées dans le cadre de ce projet. Nous détaillerons ensuite le cahier des charges et les spécifications techniques de la solution Starling, avant de décrire en profondeur la démarche DevOps adoptée et les outils mis en œuvre. La présentation de nos réalisations techniques concrètes sera suivie d'une analyse des recherches effectuées et des défis rencontrés. Enfin, nous conclurons par une synthèse des enseignements tirés et des préconisations pour l'évolution future du projet.



1. Compétences du référentiel RNCP36061 couvertes par le projet

Le titre professionnel RNCP36061 - Administrateur système DevOps est composé de trois blocs de compétences professionnelles (CCP) correspondant aux activités principales du métier. Le projet Starling a été conçu pour couvrir l'intégralité de ces compétences et en démontrer la maîtrise de manière concrète et professionnelle.

1.1. Bloc de compétences 1 : Automatiser le déploiement d'une infrastructure dans le cloud

Ce premier bloc regroupe les compétences fondamentales liées à l'automatisation et au déploiement d'infrastructures cloud. Dans le cadre du projet Starling, ces compétences ont été mises en œuvre pour créer une infrastructure AWS évolutive, sécurisée et automatisée.

1.1.1. Automatiser la création de serveurs à l'aide de scripts

La création automatisée de serveurs a été réalisée via Terraform, permettant de provisionner l'ensemble des ressources AWS nécessaires au projet Starling de manière reproductible et versionnée.

Mise en œuvre concrète :

- Création d'un cluster EKS (Elastic Kubernetes Service) avec configuration des node groups pour l'orchestration des conteneurs
- Provisionnement d'un RDS pour le stockage persistant de la base de données PostgreSQL
- Configuration des instances EC2 sous-jacentes au cluster avec dimensionnement adapté aux besoins de l'application
- Mise en place d'une architecture modulaire Terraform avec séparation des environnements (dev, preprod, prod)

1.1.2. Automatiser le déploiement d'une infrastructure

L'automatisation du déploiement complet de l'infrastructure a été réalisée via Terraform avec une approche Infrastructure as Code (IaC) permettant une gestion versionnée et collaborative.

Éléments d'infrastructure déployés automatiquement :

- VPC (Virtual Private Cloud) avec sous-réseaux publics et privés pour l'isolation réseau
- Cluster EKS avec configuration des node groups et des rôles IAM associés
- Load Balancer pour la distribution du trafic entre les pods applicatifs
- Zone DNS Route 53 pour la gestion des noms de domaine de l'application
- Security Groups configurés selon le principe du moindre privilège
- Stockage RDS pour la persistance des données PostgreSQL



DOSSIER PROJET

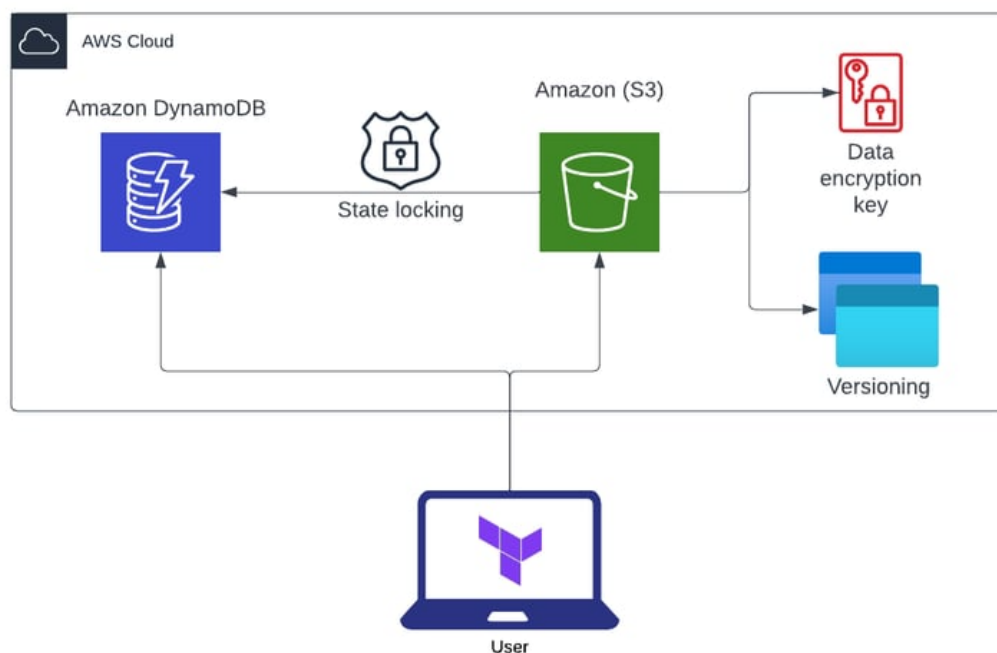
Le fichier d'état Terraform (tfstate) est stocké de manière centralisée et sécurisée dans un bucket S3 AWS (datascientst-eks) avec chiffrement activé. Une table DynamoDB (datascientest-starling-tf-lockid) assure le verrouillage du state pour éviter les conflits en cas de déploiements concurrents. Cette configuration garantit la persistance et la cohérence de l'état de l'infrastructure entre les différentes exécutions.

Le workflow de déploiement suit une stratégie simple et automatisée via GitLab CI :

Pipeline Terraform (stage: deploy) :

1. **Initialisation** : terraform init avec configuration du backend S3 et DynamoDB
2. **Validation** : terraform validate pour vérifier la syntaxe des fichiers de configuration
3. **Formatage** : terraform fmt -check pour garantir la cohérence du code (non bloquant)
4. **Planification** : terraform plan génère un plan d'exécution sauvegardé dans planfile
5. **Application conditionnelle** : terraform apply s'exécute automatiquement uniquement si :
 - La branche est main
 - L'événement est un push (pas une merge request)
 - Le plan a été généré avec succès

Diagramme Gestion du Remote State Terraform avec S3 et DynamoDB





1.1.3. Sécuriser l'infrastructure

La sécurisation de l'infrastructure constitue un enjeu majeur du projet. Plusieurs couches de sécurité ont été mises en place pour protéger l'application et les données.

Mesures de sécurité implémentées :

- Configuration des Security Groups AWS avec règles restrictives limitant l'accès aux seuls ports nécessaires
- Gestion des secrets via Kubernetes Secrets pour les credentials de base de données et les clés API
- Configuration IAM avec rôles spécifiques pour chaque service selon le principe du moindre privilège
- Chiffrement des communications via HTTPS/TLS pour tous les flux externes
- Isolation réseau des environnements (dev, preprod, prod) dans des namespaces différents
- Chiffrement de la base de donnée postgresSQL
- 2MFA impéementée

1.1.4. Mettre l'infrastructure en production dans le cloud

La mise en production de l'infrastructure suit une stratégie progressive à travers trois environnements distincts, permettant de valider chaque étape avant le déploiement en production.

Stratégie multi-environnements :

- Environnement de développement (dev) : Cluster EKS avec ressources minimales.
- Environnement de pré-production (preprod) : Réplique iso-production pour les tests de charge et de validation fonctionnelle
- Environnement de production (prod) : Infrastructure dimensionnée pour supporter la charge utilisateur réelle avec haute disponibilité
- Chaque environnement dispose de son propre namespace kubernetes
- Configuration DNS via Route 53 avec sous-domaines distincts :
 - dev.hugopradierdatascientest.com,
 - preprod.hugopradierdatascientest.com
 - prod.hugopradierdatascientest.com



1.2. Bloc de compétences 2 : Déployer en continu une application

Ce second bloc couvre les compétences liées au déploiement continu des applications. Dans le cadre de Starling, l'objectif était de mettre en place un pipeline complet permettant le déploiement automatisé de l'application Next.js (frontend) et FastAPI (backend) sur Kubernetes.

1.2.1. Préparer un environnement de test

La préparation des environnements de test constitue une étape cruciale du cycle DevOps. Trois environnements distincts ont été créés pour garantir la qualité et la stabilité avant chaque mise en production.

Configuration des environnements :

- Namespaces Kubernetes dédiés (starling-dev, starling-preprod, starling-prod) pour l'isolation des environnements
- Charts Helm paramétrables avec fichiers values distincts par environnement (values-dev.yaml, values-preprod.yaml, values-prod.yaml)
- Configuration des variables d'environnement spécifiques via ConfigMaps et Secrets Kubernetes

1.2.2. Gérer le stockage des données

La gestion du stockage persistant des données alimentaires mondiales nécessite une solution robuste et performante. PostgreSQL a été choisi pour sa fiabilité et ses capacités analytiques.

Architecture de stockage :

- Base de données PostgreSQL hébergée sur RDS garantissant la durabilité des données
- Configuration de PersistentVolumeClaims (PVC) Kubernetes pour le montage des volumes dans les pods PostgreSQL
- Stratégie de sauvegarde automatisée des données via snapshots RDS planifiés

1.2.3. Gérer des containers

La conteneurisation de l'application Starling permet d'assurer la portabilité et la reproductibilité des déploiements. L'application est composée de deux containers principaux orchestrés par Kubernetes.

Stratégie de conteneurisation :

- Container frontend : Image Docker Next.js optimisée avec build multi-stage pour réduire la taille finale
- Container backend : Image Docker FastAPI avec dépendances Python figées pour garantir la reproductibilité
- Container PostgreSQL : Image officielle PostgreSQL avec configuration personnalisée via ConfigMaps
- Registry Docker : Stockage des images dans un registry privé (Docker Hub, ECR, ou GitLab Container Registry)



- Tagging des images selon une stratégie sémantique (version + commit SHA) pour la traçabilité

1.2.4. Automatiser la mise en production d'une application avec une plateforme

L'automatisation complète du déploiement a été réalisée via une approche GitOps combinant GitLab CI pour le CI/CD et ArgoCD pour le déploiement continu sur Kubernetes.

1. Scan :

- **Objectif** : Détecter les secrets éventuellement "hardcodés" dans le code.
- **Outil utilisé** : GitLab Secret Detection.
- **Processus** : Analyse du code pour identifier et prévenir les fuites de secrets.
- **Pourquoi ?** Éviter les risques de sécurité en détectant les informations sensibles avant qu'elles ne soient déployées.

2. Build :

- **Objectif** : Construire les images Docker pour le frontend et le backend.
- **Processus** :
 - Le code source est compilé et empaqueté dans des images Docker.
 - Les images sont taguées avec le commit SHA et poussées vers l'AWS ECR.
- **Pourquoi ?** Automatiser la création des images pour garantir que le code est toujours dans un état déployable.

3. Deploy :

- **Objectif** : Déployer les images Docker sur différents environnements.
- **Processus** :
 - **Développement (dev)** : Les images sont pullées depuis l'ECR, taguées avec dev, et poussées à nouveau vers l'ECR.
 - **Préproduction (develop)** : Les images sont pullées depuis l'ECR, taguées avec preprod, et poussées à nouveau vers l'ECR.
 - **Production (main)** : Les images sont pullées depuis l'ECR, taguées avec prod et latest, et poussées à nouveau vers l'ECR. Cette étape nécessite une validation manuelle pour plus de sécurité.
- **Pourquoi ?** Faciliter le déploiement en ayant des images prêtes à être utilisées et en ajoutant une couche de validation humaine pour la production.

4. Uniformité et Sécurité :

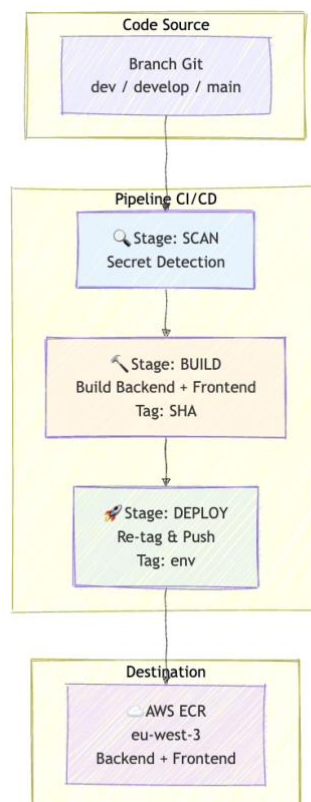
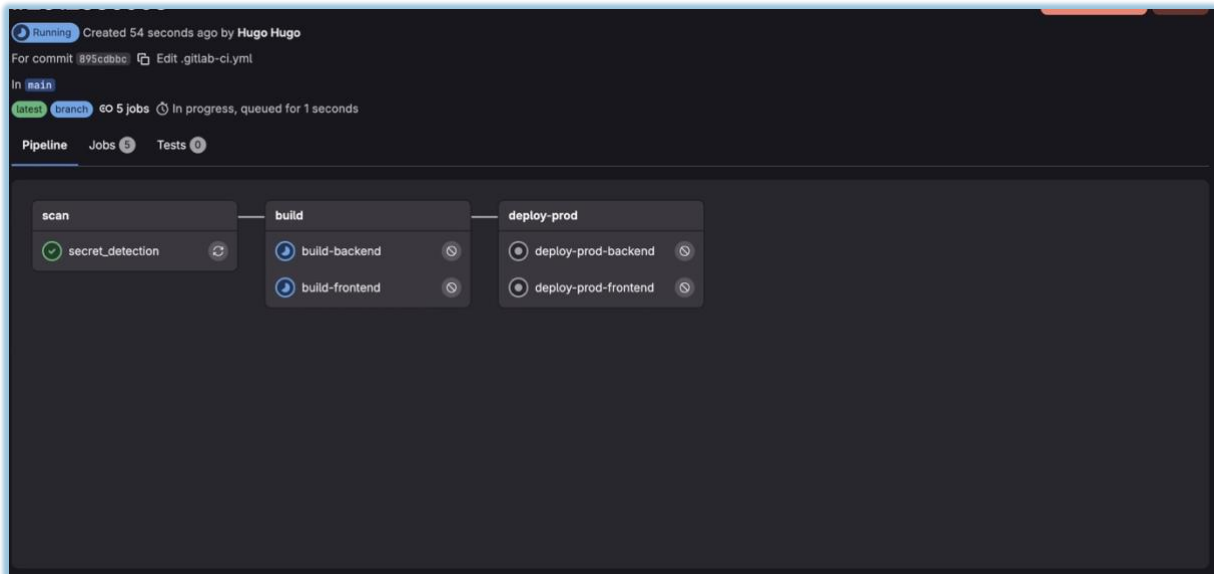
- **Uniformité** : La même stratégie est appliquée à tous les environnements (développement, préproduction, production) pour garantir la cohérence et la fiabilité.
- **Protection de la production** :
 - **Sécurité renforcée** : L'environnement de production est protégé par une étape manuelle.



DOSSIER PROJET

- **Processus** : L'utilisateur doit cliquer sur un bouton manuel pour pousser les images Docker vers la production.
- **Pourquoi ?** Ajouter une couche de validation humaine pour éviter les déploiements accidentels ou non autorisés en production.

Cette approche combine automatisation et sécurité, en s'assurant que chaque étape est validée avant de passer à la suivante, tout en protégeant l'environnement de production avec une validation manuelle.





DOSSIER PROJET

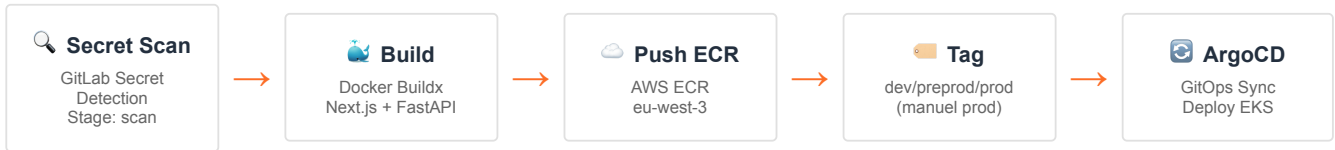
Déploiement GitOps avec ArgoCD :

- ArgoCD synchronise automatiquement l'état du cluster Kubernetes avec les manifestes Helm stockés dans Git
- Configuration d'Applications ArgoCD par environnement pointant vers les branches correspondantes du repo Helm
- Stratégie de synchronisation automatique avec self-heal activé pour corriger les dérives de configuration
- Rollback automatique en cas d'échec de déploiement via les health checks Kubernetes

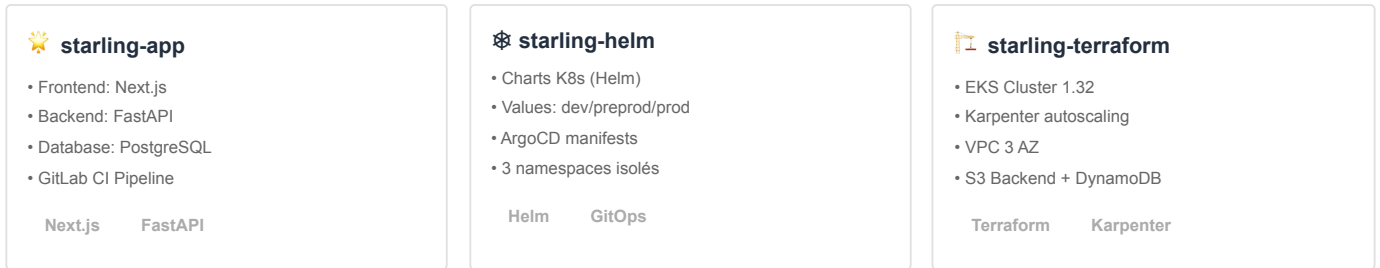
The top screenshot displays the ArgoCD interface for the 'starting-prod' application. It shows a 'Synced' status with a green heart icon and a 'Healthy' status with a green heart icon. The application is synchronized to the HEAD of the 'main' branch. The 'LAST SYNC' section indicates a successful sync on 11/29/2025 at 11:19:04. The 'APPLICATION DETAILS TREE' shows a hierarchical view of the application's components, including 'prod', 'my-secret', 'api', 'test-frontend', and 'test-backend'. The bottom screenshot shows the 'Applications' overview page. It lists three applications: 'starling-dev', 'starling-preprod', and 'starling-prod'. Each application has a 'Sync' status (Synced) and a 'Health' status (Healthy). The 'starling-prod' application is highlighted, showing its details: Project: default, Labels: default, Status: Healthy Synced, Repository: https://gitlab.com/datascientest4750052/datasci..., Target Rev: HEAD, Path: helm/prod, Destination: in-cluster, Namespace: default, Created At: 11/29/2025 09:14:53 (4 days ago), Last Sync: 12/01/2025 18:35:19 (2 days ago). The interface includes a sidebar with navigation links for Applications, Settings, User Info, and Documentation, and a top bar with a search bar and a 'Log out' button.

🌟 Projet Starling - Architecture CI/CD GitOps

📦 Pipeline CI/CD Complet

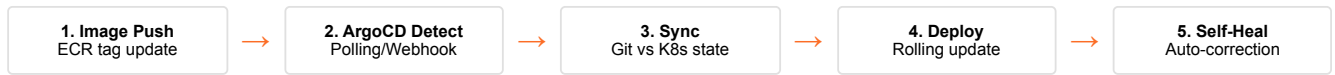


📁 Structure Git (3 Repositories)



🔄 GitOps Workflow ArgoCD

Principe: Git comme source de vérité unique (Single Source of Truth)

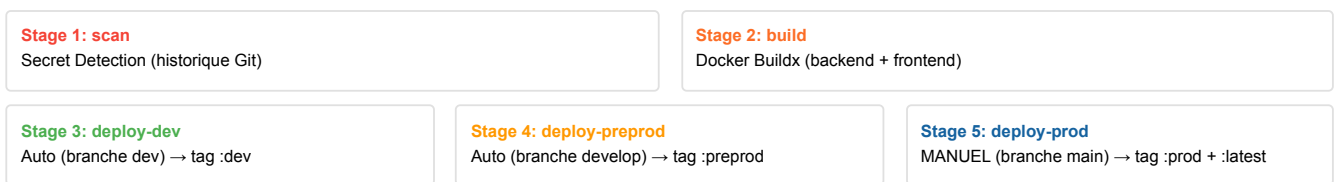


🔗 Applications: starling-dev starling-preprod starling-prod | Sync Policy: Automated + Self-Heal + Prune

🔧 Infrastructure AWS (Terraform + EKS Blueprints)



🔑 GitLab CI - Stratégie de déploiement



🔑 ECR Repositories: 479255335754.dkr.ecr.eu-west-3.amazonaws.com/starling/{backend|frontend}

⚡ Stack Technique Complète

CI/CD
GitLab CI (5 stages)

GitOps
ArgoCD + Helm

Infrastructure
Terraform + EKS Blueprints

Autoscaling
Karpenter

Container Registry
AWS ECR

Orchestration
Kubernetes 1.32

Monitoring
Prometheus + Grafana

State Backend
S3 + DynamoDB



1.3. Bloc de compétences 3 : Superviser les services déployés

Ce troisième bloc concerne la mise en place d'une observabilité complète de l'infrastructure Kubernetes et des applications déployées. L'objectif était d'assurer une visibilité en temps réel sur l'état du cluster, des ressources, et des applications pour anticiper et résoudre rapidement les incidents.

1.3.1. Architecture de supervision déployée

Une stack de monitoring complète a été mise en place combinant plusieurs solutions AWS et open-source pour couvrir l'ensemble des besoins d'observabilité :

Stack Prometheus + Grafana sur EKS

Le socle de monitoring repose sur la stack **kube-prometheus-stack** déployée via Helm Chart, qui intègre :

- **Prometheus** : Système de collecte et stockage de métriques time-series pour l'infrastructure Kubernetes
- **Grafana** : Plateforme de visualisation avec dashboards interactifs pour l'analyse des métriques
- **Node Exporter** : Agent de collecte des métriques système des nodes EC2 (CPU, RAM, disque, réseau)
- **Kube-State-Metrics** : Exposition des métriques sur l'état des objets Kubernetes (pods, deployments, services)

Cette stack fonctionne de manière autonome dans le cluster et permet une supervision détaillée de toutes les ressources Kubernetes.

Centralisation des logs avec CloudWatch

Pour la gestion centralisée des logs, **Fluent Bit** a été déployé en tant que DaemonSet sur l'ensemble des nodes du cluster. Fluent Bit collecte automatiquement les logs des conteneurs et les transmet vers **AWS CloudWatch Logs**, permettant ainsi :

- La centralisation des logs de tous les pods applicatifs
- La persistance des logs au-delà du cycle de vie des pods
- La recherche et le filtrage avancés via CloudWatch Logs Insights
- L'archivage et la rétention configurables des logs

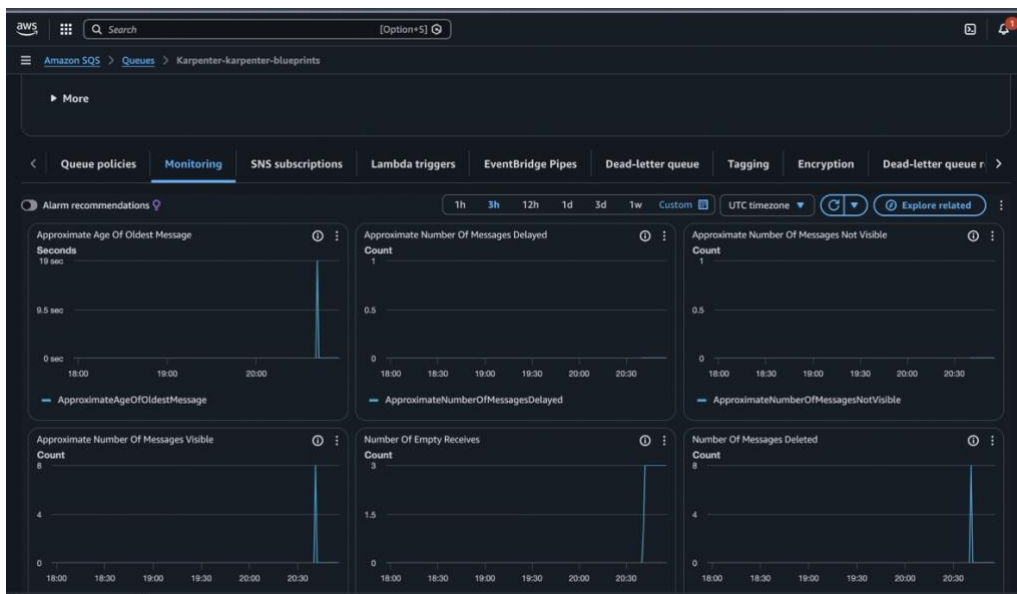


DOSSIER PROJET

Monitoring des files SQS

Pour superviser les communications asynchrones utilisées par Karpenter (autoscaler de nodes), le monitoring **Amazon SQS** a été configuré pour suivre les métriques des files de messages :

- Nombre de messages en attente
- Âge du message le plus ancien
- Messages supprimés et messages non visibles
- Taux de messages traités





1.3.2. Définir et mettre en place des statistiques de services

La définition de métriques pertinentes a été réalisée en distinguant trois niveaux de supervision :

Métriques infrastructure (niveau cluster)

Les métriques au niveau cluster permettent d'avoir une vue d'ensemble de la santé de l'infrastructure :

- **CPU et Mémoire** : Utilisation globale par namespace (kube-system, monitoring, default, karpenter)
- **Réseau** : Bande passante reçue/transmise, taux de paquets, paquets dropped par namespace
- **Pods actifs** : Nombre de pods et workloads par namespace
- **Quota et limites** : Requests et limits CPU/Memory committés par namespace



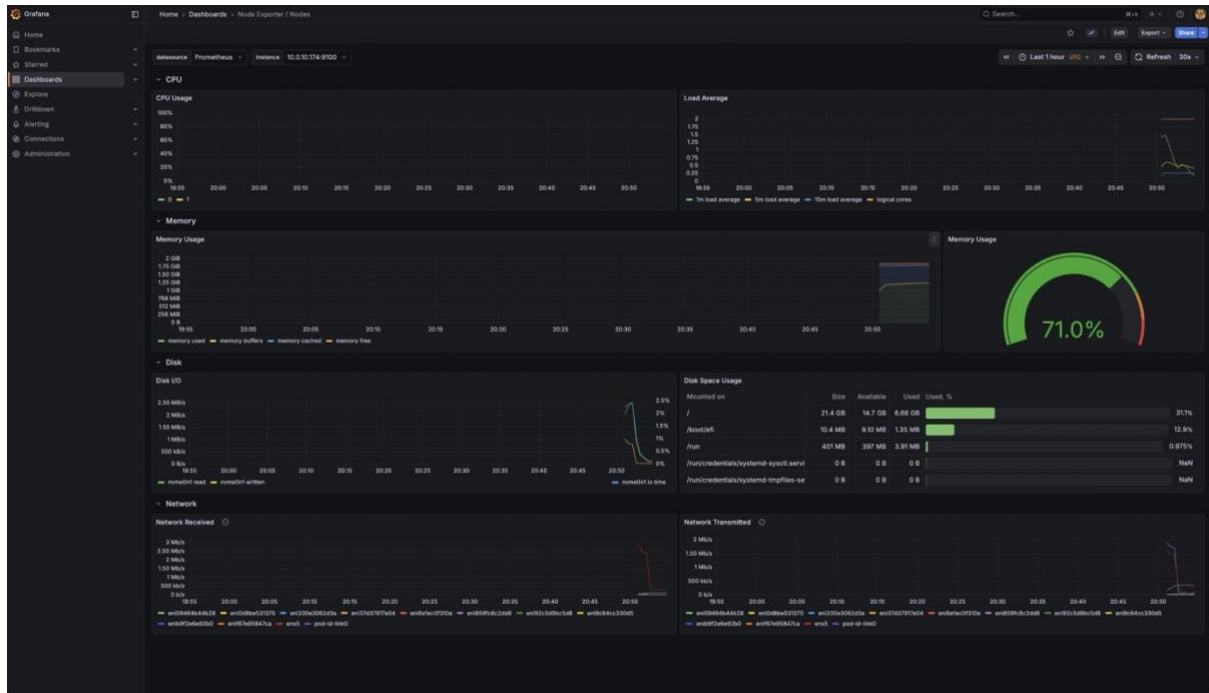


DOSSIER PROJET

Métriques nodes (niveau infrastructure)

Au niveau des nodes EC2, les métriques système sont collectées pour détecter les goulots d'étranglement :

- **CPU** : Usage par core, load average, temps idle/système/utilisateur
- **Mémoire** : RAM utilisée, buffered, cached, swap
- **Disque** : I/O read/write, espace disque utilisé par partition
- **Réseau** : Trafic entrant/sortant par interface réseau, erreurs réseau



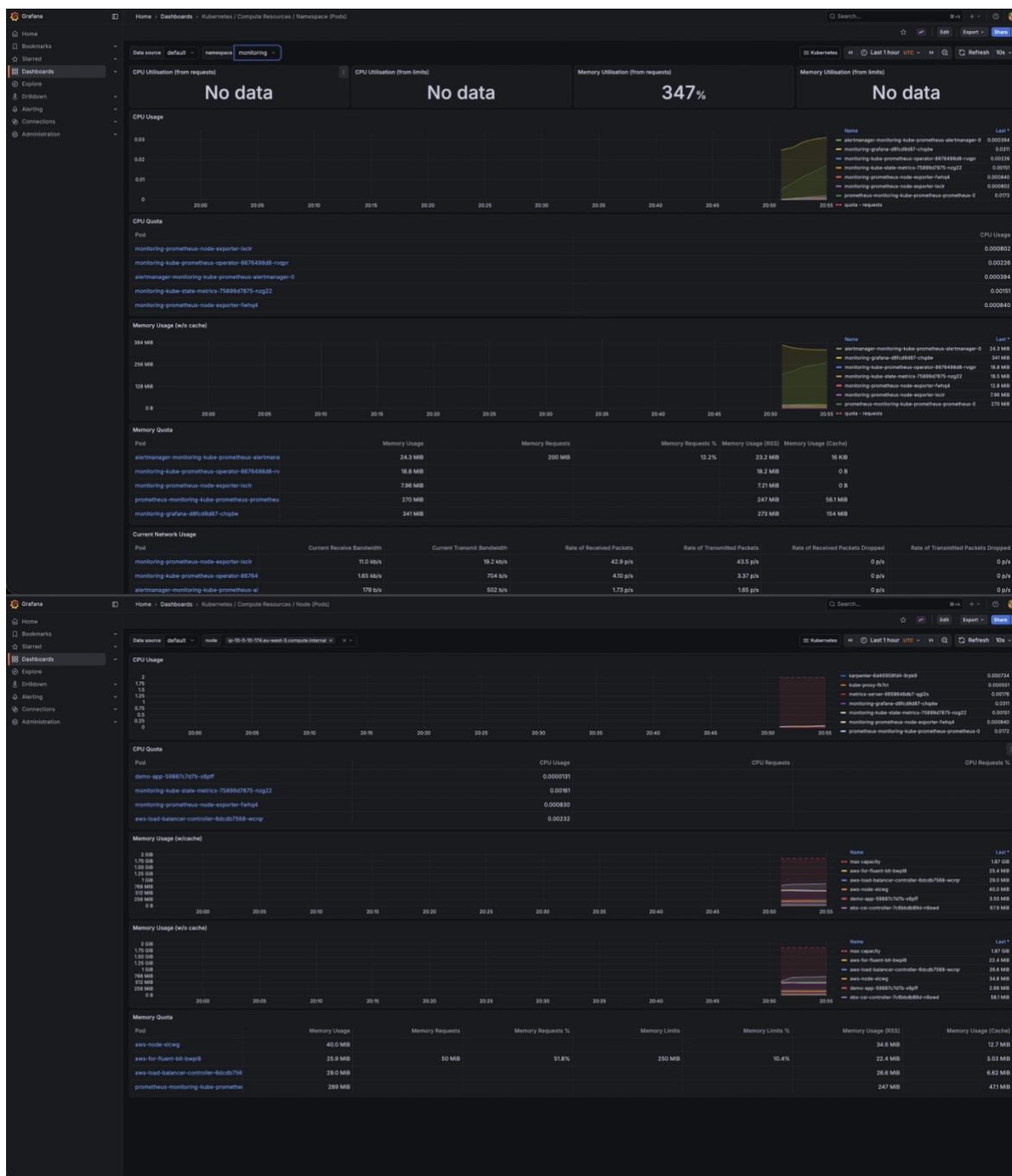


DOSSIER PROJET

Métriques applicatives (niveau pods)

Les métriques au niveau des pods permettent d'identifier les applications problématiques :

- **Utilisation CPU/Mémoire** : Consommation réelle vs requests/limits configurés
- **Redémarrages** : Nombre de restarts et crashloops des conteneurs
- **Statut** : Pods en Running, Pending, Failed, Unknown
- **Bande passante réseau** : Trafic réseau par pod



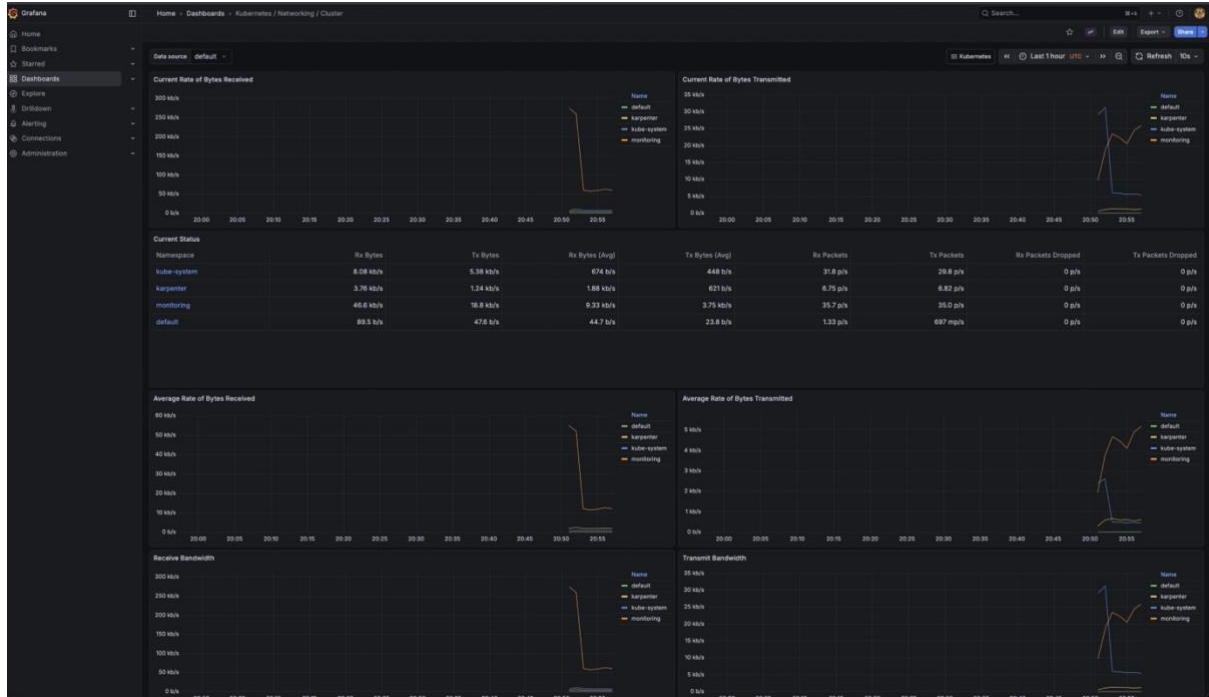


DOSSIER PROJET

Métriques réseau (niveau cluster)

Le monitoring réseau global permet d'analyser les flux de communication :

- **Trafic entrant/sortant** : Bytes reçus et transmis par namespace
- **Latence réseau** : Temps de réponse des services
- **Taux d'erreur** : Erreurs HTTP, timeouts, connexions refusées





1.3.3. Exploiter une solution de supervision

Dashboards Grafana déployés

Plusieurs dashboards pré-configurés ont été importés et personnalisés pour couvrir l'ensemble des besoins de monitoring :

Dashboard	ID Grafana	Description	Cas d'usage
Kubernetes Cluster Monitoring	3119	Vue d'ensemble du cluster avec CPU, RAM, réseau global	Supervision quotidienne, détection d'anomalies cluster-wide
Kubernetes Pods Monitoring	6417	Détail par pod : CPU, mémoire, restarts, statuts	Investigation de problèmes applicatifs, analyse de la consommation
Node Exporter Full	1860	Métriques système détaillées des nodes EC2	Diagnostic des problèmes de performance au niveau OS
Kubernetes Cluster (Prometheus)	6663	Vue cluster orientée métriques Prometheus natives	Analyse approfondie des métriques Kubernetes

Ces dashboards permettent une navigation fluide entre les différents niveaux d'abstraction : du cluster global jusqu'au pod individuel.

Utilisation opérationnelle

La stack de supervision est exploitée quotidiennement pour :

1. **Monitoring proactif** : Détection des tendances de consommation avant saturation des ressources
2. **Troubleshooting** : Analyse rapide en cas d'incident (pod crashloop, node overload, erreurs réseau)
3. **Capacity planning** : Identification des besoins de scaling basés sur l'historique de métriques
4. **Optimisation des coûts** : Détection des pods sur-dimensionnés ou sous-utilisés

Alerting (perspectives)

Bien que non implémenté durant ce stage, la stack déployée permet facilement d'ajouter :

- **Prometheus Alertmanager** : Pour les alertes sur les métriques (CPU > 80%, Memory > 90%, Pod CrashLoop)
- **CloudWatch Alarms** : Pour les alertes sur les logs patterns critiques
- Intégration avec Slack/PagerDuty pour notification en temps réel



Compétences mises en œuvre :

- Configuration et déploiement de solutions d'observabilité cloud-native (Prometheus, Grafana)
- Définition de métriques pertinentes multi-niveaux (cluster, nodes, pods, application)
- Exploitation opérationnelle des dashboards pour le troubleshooting et l'optimisation
- Mise en place de collecte centralisée de logs avec Fluent Bit et CloudWatch



1.3.3. Échanger sur des réseaux professionnels éventuellement en anglais

La réalisation du projet Starling a nécessité de nombreuses recherches et échanges avec la communauté DevOps, majoritairement en anglais. Cette compétence est essentielle pour résoudre les problèmes techniques complexes et rester à jour sur les bonnes pratiques.

Ressources documentaires consultées :

- Documentation officielle AWS (EKS, RDS, Route 53, IAM) en anglais
- Documentation Kubernetes, Helm, ArgoCD et Terraform
- GitHub Issues et Discussions des projets open-source utilisés
- Stack Overflow pour la résolution de problèmes techniques spécifiques

Échanges avec la communauté :

- Consultation des forums Reddit (r/devops, r/kubernetes) pour les bonnes pratiques
- Participation aux discussions GitHub pour signaler des bugs ou proposer des améliorations
- Veille technologique via blogs techniques (AWS Blog, CNCF, DevOps.com) principalement en anglais



1.4. Synthèse de la couverture des compétences RNCP36061

Le projet Starling a permis de couvrir l'intégralité des compétences du référentiel RNCP36061 - Administrateur système DevOps. Le tableau ci-dessous synthétise cette couverture :

Bloc de compétences	Compétence	Mise en œuvre dans Starling
BC01 - Automatiser le déploiement	Automatiser la création de serveurs	Terraform + EKS + RDS
	Automatiser le déploiement	Infrastructure AWS complète (VPC, EKS, Route 53)
	Sécuriser l'infrastructure	IAM, Security Groups, Secrets, chiffrement
	Mettre en production	3 environnements (dev/preprod/prod)
BC02 - Déployer en continu	Préparer environnements de test	Namespaces K8s + Charts Helm paramétrables
	Gérer le stockage	PostgreSQL sur RDS
	Gérer des containers	Docker (Next.js + FastAPI) + Registry
	Automatiser la mise en production	GitLab CI + ArgoCD (GitOps)
BC03 - Superviser	Définir des statistiques	Métriques infra/app (CPU, RAM)
	Exploiter une solution de supervision	Prometheus + Grafana
	Échanger en anglais	Docs AWS/K8s, Stack Overflow, GitHub

Cette couverture exhaustive des compétences RNCP36061 démontre que le projet Starling constitue un cas d'usage complet et professionnel, permettant de valider l'ensemble des acquis nécessaires à l'exercice du métier d'Administrateur système DevOps.



2. Cahier des charges du projet Starling

2.1. Contexte et genèse du projet

Le projet Starling trouve son origine dans un projet data antérieur réalisé par un partenaire de formation chez DataScientest. Face à l'importance des enjeux de sécurité alimentaire mondiale et constatant la difficulté d'accès à des données agrégées et visualisées de manière accessible, l'idée d'une plateforme de visualisation dédiée a émergé.

Dans le cadre de ma formation Ingénieur DevOps chez DataScientest (promotion Novembre 2024, démarrage mi-2025), ce projet a été choisi comme projet fil rouge. L'équipe initiale était composée de trois personnes, mais suite au départ précoce de mes deux partenaires durant la phase de développement, j'ai poursuivi et mené à bien l'intégralité du projet de manière individuelle.

L'enjeu principal du projet n'était pas tant le développement de l'application elle-même, mais bien la démonstration d'une **démarche DevOps complète et professionnelle** : automatisation de l'infrastructure, CI/CD, orchestration de containers, stratégie GitOps, monitoring et sécurité.

2.2. Objectifs du projet

Objectifs DevOps et techniques (prioritaires) :

- Automatiser le provisionnement complet de l'infrastructure AWS via Terraform
- Déployer l'application sur un cluster Kubernetes managé (Amazon EKS)
- Mettre en place un pipeline CI/CD complet avec GitLab CI
- Implémenter une stratégie GitOps avec ArgoCD
- Assurer l'observabilité via Prometheus et Grafana
- Gérer trois environnements distincts (développement, pré-production, production)
- Sécuriser l'infrastructure et les données

2.3. Contraintes du projet

Contraintes organisationnelles :

- Projet initialement prévu en équipe de trois personnes
- Départ précoce des deux partenaires durant la phase de développement, nécessitant une réorganisation complète
- Problèmes de santé personnels ayant impacté la planification
- Durée limitée de la formation bootcamp

Contraintes budgétaires :

- Budget AWS alloué au projet : 400€ pour l'ensemble de l'infrastructure
- Nécessité d'optimiser les coûts cloud en permanence
- Monitoring des coûts AWS via Cost Explorer



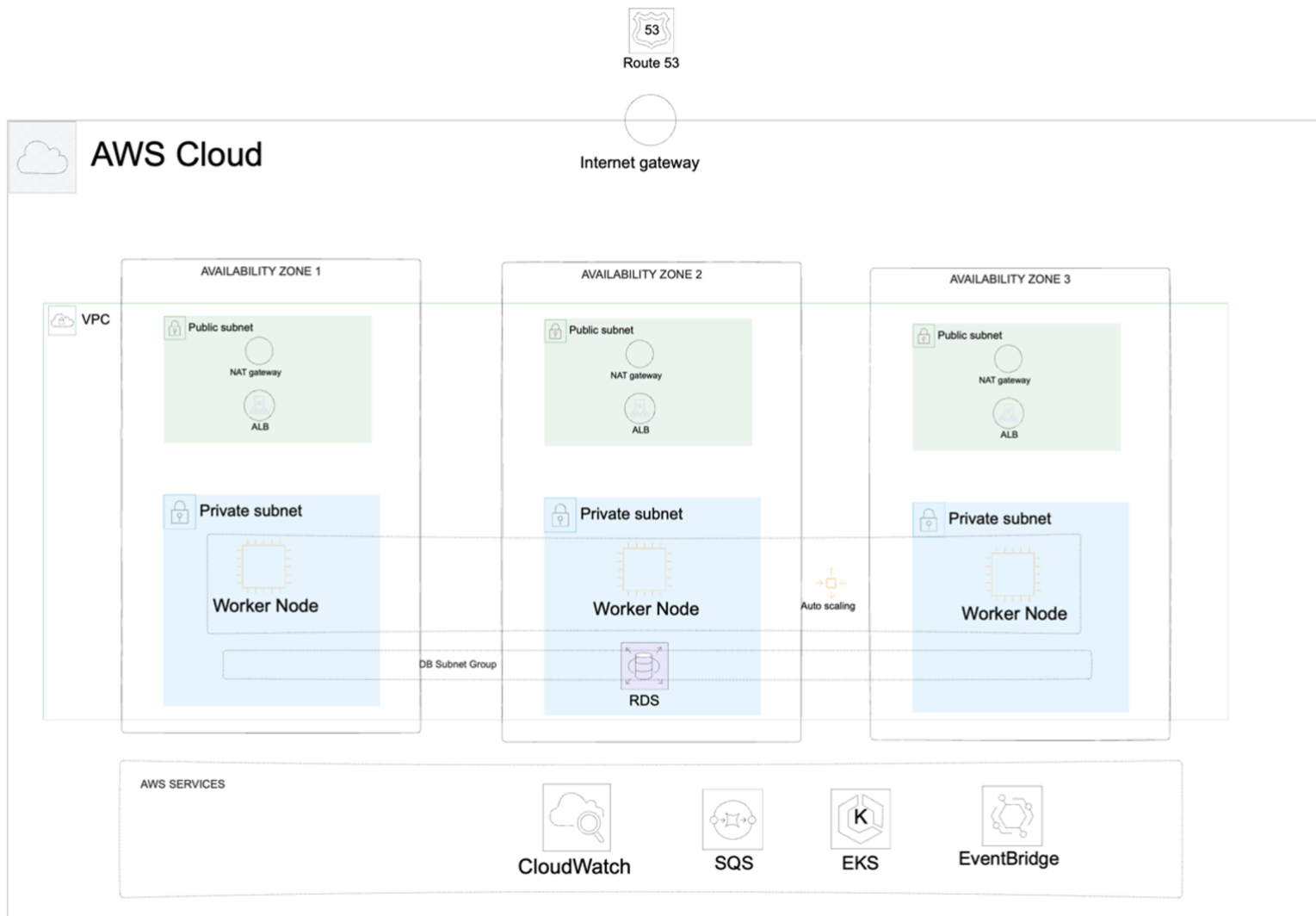
2.4. Livrables attendus

- Application Starling déployée et fonctionnelle en production
- Infrastructure AWS automatisée via Terraform
- Trois repositories GitHub organisés (app, terraform, helm)
- Pipeline CI/CD GitLab fonctionnel
- ArgoCD configuré pour le GitOps
- Stack Prometheus/Grafana déployée
- Documentation technique complète



DOSSIER PROJET

Diagramme de l'infrastructure AWS attendue :





3. Description de la démarche et des outils utilisés

3.1. Organisation du projet et évolution de l'équipe

3.1.1. Composition initiale et répartition des tâches

Le projet Starling a démarré avec une équipe de **trois personnes** au sein de la promotion DataScientest. La répartition initiale des responsabilités s'articulait autour des compétences de chacun :

Premier membre (départ précoce) :

- Responsable de la récupération et de l'intégration du projet data antérieur (données alimentaires mondiales issues de sa formation Data Science précédente chez DataScientest)
- Création de la base de données PostgreSQL initiale avec les datasets FAO
- Départ du projet durant la phase de cadrage initiale

Second membre :

- Responsable du développement de l'API backend FastAPI
- Connexion de l'API à la base de données PostgreSQL
- Départ du projet durant la phase de développement de la version EC2

Hugo PRADIER (moi-même) :

- Responsable de la création du frontend Next.js
- Évolution vers la responsabilité complète du projet après le départ des partenaires
- Prise en charge de l'infrastructure, du déploiement et de l'automatisation DevOps

3.1.2. Difficultés organisationnelles et réorganisation

Le départ successif des deux partenaires de formation a nécessité une **réorganisation complète du projet**. Initialement prévu comme un travail d'équipe collaboratif, le projet est devenu un travail individuel nécessitant une montée en compétences rapide sur l'ensemble de la stack technique.

Cette contrainte s'est révélée être également une opportunité : elle m'a permis d'acquérir une **vision d'ensemble complète** du projet, de la base de données jusqu'au monitoring, en passant par l'infrastructure et le CI/CD.



3.2. Démarche technique : de la version EC2 à la version EKS

3.2.1. Phase 1 : Version initiale sur EC2 (avec l'équipe)

Dans un premier temps, l'équipe a opté pour une approche simplifiée afin de valider rapidement la faisabilité technique du projet. Cette première version reposait sur une architecture minimaliste :

Infrastructure initiale :

- Instance EC2 AWS de taille modeste (budget limité)
- Déploiement d'un cluster Kubernetes léger (mini-cluster)
- Pipeline CI/CD basique pour automatiser les déploiements
- PostgreSQL hébergé directement sur l'instance EC2

Difficultés rencontrées :

- **Limitation de ressources** : La taille de l'instance EC2 était insuffisante pour supporter l'ensemble de la stack applicative et des outils de monitoring
- **Instabilité du cluster** : L'installation de composants additionnels (Prometheus, Grafana) provoquait régulièrement des crashes et des redémarrages du cluster
- **Absence d'isolation des environnements** : Tous les composants (dev, monitoring, application) cohabitaient sur la même instance, rendant le débogage complexe

Enseignements tirés : Cette première phase a permis de valider l'architecture applicative (Next.js + FastAPI + PostgreSQL) et d'identifier les limites d'une approche mono-instance. Elle a également mis en évidence la nécessité d'une infrastructure scalable et professionnelle pour répondre aux exigences du référentiel RNCP36061.



3.2.2. Phase 2 : Reconstruction complète sur Amazon EKS (solo)

Suite au départ de mon second partenaire et en vue de la soutenance finale devant le jury de certification, j'ai pris la décision de **reconstruire l'intégralité de l'infrastructure** en adoptant une approche DevOps professionnelle et industrialisée.

Étape 1 : Infrastructure as Code avec Terraform

Le projet a démarré par la définition complète de l'infrastructure AWS via Terraform. Pour accélérer le développement et garantir les meilleures pratiques de sécurité, j'ai utilisé les **AWS Blueprints for Terraform** (repository GitHub terraform-aws-eks), qui fournissent des modules Terraform pré-configurés et validés par AWS pour le déploiement de clusters EKS.

Choix technique : Karpenter

L'un des choix structurants de cette architecture a été l'intégration de **Karpenter** pour la gestion automatique des nœuds du cluster EKS. Ce choix repose sur mon expérience acquise lors de mon stage DevOps précédent chez Exoscale, où j'avais déjà travaillé avec cet outil. Karpenter permet :

- Un **autoscaling dynamique** des nœuds en fonction de la charge réelle des pods
- Une **optimisation des coûts** AWS grâce à la sélection automatique des types d'instances EC2 les plus adaptés
- Une **réduction du temps de provisionnement** des nœuds par rapport aux node groups classiques

Étape 2 : Déploiement de l'application sur Kubernetes

Une fois l'infrastructure provisionnée, j'ai procédé au déploiement de l'application Starling sur le cluster EKS :

- Création de **namespaces dédiés** pour chaque environnement (starling-dev, starling-preprod, starling-prod)
- Packaging de l'application via **Helm charts** avec paramétrage par environnement
- Configuration des **PersistentVolumeClaims** pour le stockage PostgreSQL sur EBS
- Mise en place des **Services** et **Ingress** Kubernetes pour l'exposition de l'application



Étape 3 : Automatisation du déploiement (CI/CD et GitOps)

La phase finale a consisté à automatiser complètement le cycle de déploiement :

- **Pipeline GitLab CI** : Build, test, création des images Docker, push vers AWS ECR
- **ArgoCD** : Synchronisation automatique des déploiements Kubernetes depuis Git (approche GitOps)
- **Stratégie de déploiement progressive** : dev automatique, preprod après validation, prod avec approbation manuelle

Étape 4 : Observabilité (Prometheus et Grafana)

La dernière brique du projet a été l'implémentation de la stack de monitoring :

- Déploiement de **Prometheus** via le chart Helm kube-prometheus-stack
- Configuration de **Grafana** avec dashboards personnalisés pour Starling
- Mise en place d'**alertes** critiques via Alertmanager

3.3. Outils de collaboration et méthodes de travail

3.3.1. Phase collaborative (équipe de 3 puis 2 personnes)

Durant la phase initiale du projet, l'équipe a utilisé les outils de communication suivants :

- **Slack** : Communication écrite quotidienne, partage de documentation, questions techniques
- **Discord** : Visioconférences hebdomadaires pour les points d'avancement et les décisions techniques
- **Git** : Versionnement du code avec branches par fonctionnalité et merge requests
- **Notion** : Documentation partagée et suivi des tâches (to-do lists)

La méthodologie adoptée était inspirée des pratiques agiles :

- **Sprints d'une semaine** avec objectifs définis en début de sprint
- **Rétrospectives** hebdomadaires pour identifier les difficultés et ajuster la stratégie
- **Répartition des tâches** selon les compétences de chacun



3.3.2. Phase solo avec accompagnement mentor

Après le départ de mes partenaires, la collaboration s'est concentrée sur un accompagnement individuel avec **Benjamin Fiche**, mon mentor projet chez DataScientest :

- **Calls hebdomadaires d'une heure** sur Slack pour revue de l'avancement et déblocage technique
- **Échanges asynchrones** sur Slack pour les questions ponctuelles
- **Revue de code** et validation des choix d'architecture

Cet accompagnement a été déterminant pour :

- Maintenir la **motivation** dans un contexte de projet solo
- Valider les **choix techniques** (Karpenter, ArgoCD, stratégie de déploiement)
- Débloquent des **situations complexes** (configuration IAM, gestion du state Terraform, optimisation des coûts AWS)
- Préparer la **soutenance finale** devant le jury de certification

3.3.3. Veille technologique et communauté DevOps

En complément de l'accompagnement mentor, j'ai largement sollicité la communauté DevOps open-source :

- **Documentation officielle** (AWS, Kubernetes, Terraform, ArgoCD) en anglais
- **Stack Overflow** pour résoudre des problèmes techniques spécifiques
- **Reddit** (r/devops, r/kubernetes) pour découvrir les bonnes pratiques
- **GitHub Issues** des projets open-source pour signaler des bugs et proposer des améliorations
- **Blogs techniques** (AWS Blog, CNCF) pour la veille sur les nouvelles fonctionnalités

3.4. Synthèse de la démarche

La démarche adoptée dans le projet Starling illustre une progression itérative typique d'un projet DevOps :

1. **Prototype rapide** (EC2) pour valider la faisabilité → Échec contrôlé
2. **Reconstruction professionnelle** (EKS + Terraform) avec les bonnes pratiques → Succès
3. **Industrialisation** (CI/CD + GitOps + Monitoring) pour un déploiement automatisé → Production

Cette approche, bien que contrainte par les départs successifs de mes partenaires, a permis d'acquérir une **expertise complète** sur l'ensemble de la chaîne DevOps et de démontrer ma capacité à **gérer un projet d'infrastructure de bout en bout** de manière autonome.



4. Réalisations du candidat

Cette section présente les scripts et configurations les plus significatifs du projet Starling, en argumentant les choix techniques effectués et en démontrant la maîtrise des outils DevOps mis en œuvre.

4.1. Infrastructure as Code : Configuration Terraform du cluster EKS

4.1.1. Présentation et objectif

Le fichier main.tf constitue le **cœur de l'infrastructure** du projet Starling. Il définit de manière déclarative l'ensemble des ressources AWS nécessaires au déploiement de l'application : cluster EKS, VPC, sous-réseaux, groupes de sécurité, et intégration de Karpenter pour l'autoscaling intelligent.

Emplacement : terraform/main.tf

Lignes de code : ~250 lignes

Technologies : Terraform 1.5+, AWS Provider, modules officiels AWS

4.1.2. Choix techniques majeurs

Utilisation des modules Terraform officiels

```
hcl

module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "20.37.0"
  # ...
}

module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.21.0"
  # ...
}
```

Justification : Les modules Terraform officiels maintenus par la communauté AWS garantissent :

- **Bonnes pratiques de sécurité** intégrées (IAM roles, Security Groups)
- **Mises à jour régulières** pour compatibilité avec les nouvelles versions Kubernetes
- **Réduction du code** à écrire (abstraction de la complexité)
- **Documentation exhaustive** et support communautaire



4.1.3. Architecture réseau sécurisée (VPC)

```
hcl

module "vpc" {
  name = local.name
  cidr = "10.0.0.0/16"

  azs          = slice(data.aws_availability_zones.available.names, 0, 3)
  private_subnets = [for k, v in local.azs : cidrsubnet(local.vpc_cidr, 4, k)]
  public_subnets  = [for k, v in local.azs : cidrsubnet(local.vpc_cidr, 8, k + 48)]

  enable_nat_gateway  = true
  single_nat_gateway  = true
  enable_dns_hostnames = true
}
```

Points clés :

- **Isolation réseau** : Sous-réseaux privés pour les nodes EKS, publics pour les Load Balancers
- **Haute disponibilité** : Répartition sur 3 zones de disponibilité (AZ) pour tolérance aux pannes
- **NAT Gateway unique** : Optimisation des coûts (60\$/mois économisés vs 3 NAT Gateways)
- **Tagging Kubernetes** : Tags spécifiques pour découverte automatique par les Load Balancers

4.1.4. Cluster EKS avec addons essentiels

```
hcl

cluster_addons = {
  aws-ebs-csi-driver = { most_recent = true }
  coredns             = { most_recent = true }
  kube-proxy          = { most_recent = true }
  vpc-cni = {
    most_recent = true
    configuration_values = jsonencode({
      env = {
        ENABLE_PREFIX_DELEGATION = "true"
        WARM_PREFIX_TARGET       = "1"
      }
    })
  }
}
```



Justification :

- **EBS CSI Driver** : Requis pour le provisionnement des volumes persistants PostgreSQL
- **VPC-CNI avec prefix delegation** : Augmente le nombre d'IPs disponibles par node (optimisation réseau)
- **CoreDNS et kube-proxy** : Composants essentiels pour la résolution DNS et le réseau Kubernetes

4.1.5. Karpenter pour l'autoscaling intelligent

```
hcl
eks_managed_node_groups = {
  mng = {
    instance_types = ["t3.small"]
    max_size       = 2
    desired_size    = 2
    min_size       = 2

    labels = {
      "karpenter.sh/controller" = "true"
    }
  }
}
```

Stratégie hybride :

- **Node group managé** (2x t3.small) : Réserve exclusivement pour Karpenter et les composants système
- **Nodes Karpenter** : Provisionnés dynamiquement pour les workloads applicatifs (Starling)

Avantages de Karpenter :

- **Réduction des coûts** : Provisionne uniquement les ressources nécessaires
- **Réactivité** : Scaling en ~30 secondes vs 3-5 minutes pour Cluster Autoscaler
- **Optimisation automatique** : Sélection des types d'instances EC2 les plus adaptés



4.1.6. Intégration AWS Load Balancer Controller

```
hcl

module "eks_blueprints_addons" {
  source  = "aws-ia/eks-blueprints-addons/aws"
  version = "1.21.0"

  enable_aws_load_balancer_controller = true
}
```

Rôle : Gestion automatique des Application Load Balancers (ALB) et Network Load Balancers (NLB) via les ressources Kubernetes Ingress et Service.

4.1.7. Gestion des identités avec EKS Pod Identity

```
hcl

module "aws_ebs_csi_pod_identity" {
  source  = "terraform-aws-modules/eks-pod-identity/aws"
  version = "1.12.0"

  attach_aws_ebs_csi_policy = true

  association_defaults = {
    namespace      = "kube-system"
    service_account = "ebs-csi-controller-sa"
  }
}
```

Sécurité : Utilisation de Pod Identity (successeur d'IRSA) pour donner au CSI Driver EBS les permissions IAM nécessaires sans credentials statiques.



4.2. Pipeline CI/CD : Automatisation GitLab CI

4.2.1. Présentation et objectif

Le fichier `.gitlab-ci.yml` orchestre l'ensemble du cycle de vie de l'application Starling, du scan de sécurité jusqu'au déploiement en production. Ce pipeline garantit la **qualité**, la **sécurité** et la **traçabilité** de chaque version déployée.

Emplacement : `.gitlab-ci.yml` (racine du repository application)

Lignes de code : ~400 lignes

Technologies : GitLab CI/CD, Docker Buildx, AWS ECR, AWS CLI

4.2.2. Architecture du pipeline : 5 stages

```
yml
stages:
  - scan
  - build
  - deploy-dev
  - deploy-preprod
  - deploy-prod
```

Cette organisation séquentielle garantit qu'aucune image Docker n'est construite sans validation de sécurité, et qu'aucun déploiement n'intervient sans build réussi.



4.2.3. Stage 1 : Scan de sécurité (Secret Detection)

```
yaml
secret_detection:
  stage: scan
  image: "$CI_TEMPLATE_REGISTRY_HOST/security-products/secrets:5"
  variables:
    SECRET_DETECTION_HISTORIC_SCAN: 'true'
  artifacts:
    reports:
      secret_detection: gl-secret-detection-report.json
```

Objectif : Détecter les secrets (API keys, tokens, passwords) accidentellement commités dans le code source.

Fonctionnement :

- **Scan historique** : Analyse l'ensemble de l'historique Git (pas seulement le dernier commit)
- **Rapport JSON** : Génère un artifact GitLab consultable dans l'interface
- **Non-bloquant** (allow_failure: true) : Évite de bloquer le pipeline en cas de faux positifs

Justification : Cette étape est **critique** pour la sécurité. Un secret exposé publiquement sur GitLab peut être exploité en quelques minutes par des bots automatisés.

4.2.4. Stage 2 : Build des images Docker (Backend & Frontend)

Configuration Docker Buildx multi-plateforme

```
yaml
before_script:
  # Installation de Docker Buildx
  - mkdir -p ~/.docker/cli-plugins
  - curl -SL https://github.com/docker/buildx/releases/download/v0.12.0/buildx-v0.12.0-linux-$(uname -m).tar.gz -o ~/.docker/cli-plugins/docker-buildx
  - chmod +x ~/.docker/cli-plugins/docker-buildx

  # Configuration QEMU pour multi-plateforme
  - docker run --privileged --rm tonistiigi/binfmt --install all
  - docker buildx create --use --name multiarch --driver docker-container
```



Justification technique :

- **Docker Buildx** : Permet la compilation pour différentes architectures (amd64, arm64) depuis un seul pipeline
- **QEMU** : Émulation des architectures cibles pour compatibilité multi-plateforme
- **Driver docker-container** : Isole les builds dans un container dédié pour éviter les conflits

Authentification ECR et build

```
yaml
script:
- aws ecr get-login-password --region $AWS_DEFAULT_REGION |
  docker login --username AWS --password-stdin 479255335754.dkr.ecr.$AWS_DEFAULT_REI
- cd backend
- docker buildx build --platform linux/amd64
  -f ./dockerfile/fastapi.Dockerfile
  -t $ECR_BACKEND:$CI_COMMIT_SHORT_SHA --push .
```

Bonnes pratiques :

- **Tagging avec SHA Git** : \$CI_COMMIT_SHORT_SHA permet une traçabilité parfaite entre code et image
- **Push direct vers ECR** : --push évite un stockage intermédiaire local
- **Plateforme spécifique** : --platform linux/amd64 pour compatibilité EKS (nodes EC2 x86)



4.2.5. Stages 3, 4, 5 : Déploiements progressifs (Dev, Preprod, Prod)

Stratégie de déploiement par environnement

```
yaml
# DEV : Automatique sur branche 'dev'
deploy-dev-backend:
  stage: deploy-dev
  script:
    - docker pull $ECR_BACKEND:$CI_COMMIT_SHORT_SHA
    - docker tag $ECR_BACKEND:$CI_COMMIT_SHORT_SHA $ECR_BACKEND:dev
    - docker push $ECR_BACKEND:dev
  only:
    - dev

# PREPROD : Automatique sur branche 'develop'
deploy-preprod-backend:
  only:
    - develop

# PROD : Manuel sur branche 'main'
deploy-prod-backend:
  stage: deploy-prod
  only:
    - main
  when: manual # ⚠ Approbation humaine requise
```

Architecture Git Flow :

- **Branche dev** → Déploiement automatique en dev (tests rapides)
- **Branche develop** → Déploiement automatique en preprod (validation fonctionnelle)
- **Branche main** → Déploiement **manuel** en production (protection)

Justification du déploiement manuel en prod :

- **Contrôle humain** : Évite les déploiements accidentels en production
- **Validation métier** : Permet une revue finale avant la mise en production
- **Gestion des rollbacks** : Facilite l'identification de la version déployée

Double tagging en production



DOSSIER PROJET

yaml

```
deploy-prod-backend:
  script:
    - docker tag $ECR_BACKEND:$CI_COMMIT_SHORT_SHA $ECR_BACKEND:prod
    - docker tag $ECR_BACKEND:$CI_COMMIT_SHORT_SHA $ECR_BACKEND:latest
    - docker push $ECR_BACKEND:prod
    - docker push $ECR_BACKEND:latest
```

Stratégie de tagging :

- **Tag prod** : Référence stable pour l'environnement de production
- **Tag latest** : Pointe toujours vers la dernière version déployée en production
- **Tag \$CI_COMMIT_SHORT_SHA** : Conservé pour traçabilité et rollback potentiel



5. DESCRIPTION D'UNE SITUATION DE TRAVAIL AYANT NÉCESSITÉ UNE RECHERCHE

5.1. Contexte de la problématique

Lors de la phase de reconstruction complète de l'infrastructure Starling sur Amazon EKS, j'ai été confronté à une **problématique majeure** : la complexité de déployer un cluster Kubernetes production-ready avec toutes les bonnes pratiques de sécurité, de performance et de scalabilité.

Situation initiale :

- Besoin d'un cluster EKS avec autoscaling intelligent (Karpenter)
- Nécessité d'intégrer de multiples add-ons (EBS CSI Driver, Load Balancer Controller, CoreDNS, VPC-CNI)
- Exigences de sécurité strictes (IAM roles, Security Groups, isolation réseau)
- Contrainte de temps pour la maintenance
- **Manque d'expérience** sur la configuration complète d'un cluster EKS en production

Ma première approche consistait à créer le cluster EKS "from scratch" en écrivant manuellement l'ensemble du code Terraform. Cependant, après plusieurs jours de travail, je me heurtais à des erreurs de configuration récurrentes :

- Incompatibilités entre versions d'add-ons Kubernetes
- Configuration IAM incorrecte pour Karpenter (permissions manquantes)
- Problèmes de découverte réseau pour les Load Balancers
- Difficulté à intégrer le VPC-CNI avec prefix delegation

5.2. Démarche de recherche

Face à ces blocages, j'ai entrepris une démarche de recherche méthodique pour trouver une solution fiable et professionnelle.

5.2.1. Sources consultées initialement

Documentation officielle AWS EKS :

- Parcours de la documentation docs.aws.amazon.com/eks/ pour comprendre les concepts fondamentaux
- Étude des guides "Getting Started" et "Best Practices"
- **Limite** : Documentation très théorique, manque d'exemples concrets de déploiement complet

Forums et communautés :

- Stack Overflow : Recherche de questions similaires sur les erreurs rencontrées



- Reddit r/kubernetes et r/aws : Consultation des discussions sur les déploiements EKS
- **Limite** : Réponses fragmentées, pas de vision d'ensemble pour une architecture complète

Modules Terraform Community :

- Exploration du module terraform-aws-modules/eks/aws
- Étude des exemples fournis dans le repository GitHub
- **Limite** : Exemples basiques, ne couvrant pas l'intégration Karpenter et les addons avancés

5.2.2. Découverte de la solution : AWS EKS Blueprints

Après plusieurs jours de recherches infructueuses, je suis tombé sur l'article de blog AWS suivant :

"Bootstrapping clusters with EKS Blueprints"

URL : <https://aws.amazon.com/fr/blogs/containers/bootstrapping-clusters-with-eks-blueprints/>

Cet article présentait le projet **AWS EKS Blueprints for Terraform**, une initiative open-source maintenue par AWS qui fournit des **configurations Terraform pré-configurées** pour déployer des clusters EKS production-ready avec toutes les bonnes pratiques intégrées.

Éléments décisifs de cet article :

- Présentation d'architectures de référence validées par AWS
- Exemples concrets avec Karpenter pré-configuré
- Intégration des addons essentiels (EBS CSI, Load Balancer Controller, etc.)
- Code Terraform modulaire et réutilisable
- Repository GitHub public avec documentation complète

5.3. Implémentation de la solution

5.3.1. Exploration du repository EKS Blueprints

Suite à la lecture de l'article, j'ai exploré le repository GitHub officiel :

Repository : github.com/aws-ia/terraform-aws-eks-blueprints

Ce repository contenait plusieurs exemples ("patterns") de déploiement EKS, dont un spécifiquement dédié à Karpenter : `patterns/karpenter/`



Avantages identifiés :

- Code Terraform modulaire et bien structuré
- Configuration IAM complète pour Karpenter (rôles, politiques, IRSA)
- Intégration du module eks-blueprints-addons pour déployer automatiquement les addons
- Bonnes pratiques de sécurité appliquées (Security Groups, Pod Identity)
- Tagging réseau correct pour découverte automatique par Kubernetes

5.3.2. Adaptation au projet Starling

J'ai utilisé cet exemple comme **base de départ**, puis l'ai adapté aux besoins spécifiques de Starling :

Modifications apportées :

- Ajustement du CIDR du VPC (10.0.0.0/16 adapté au nombre de pods attendu)
- Réduction de la taille du node group managé (t3.small au lieu de t3.medium pour optimisation des coûts)
- Configuration du backend Terraform S3 pour stockage du state
- Ajout de tags personnalisés pour traçabilité (blueprint = "karpenter-blueprints")
- Suppression des addons non nécessaires (FluentBit configuré mais non critique pour la phase initiale)



6. CONCLUSION ET OUVERTURE

6.1. Synthèse du projet et réponse aux problématiques

Le projet Starling avait pour ambition de démontrer la maîtrise d'une démarche DevOps complète appliquée à une problématique concrète : la **démocratisation de l'accès aux données alimentaires mondiales**. Au-delà de l'enjeu applicatif, l'objectif principal était de couvrir l'intégralité des compétences du référentiel RNCP36061 - Administrateur système DevOps.

Résultats obtenus :

-  **Application fonctionnelle en production** : Plateforme web accessible permettant la visualisation de 3 indicateurs de sécurité alimentaire (aide internationale, sous-nutrition, utilisation des céréales)
-  **Infrastructure automatisée** : Cluster EKS déployé via Terraform avec Karpenter, VPC sécurisé, stockage RDS pour PostgreSQL
-  **Pipeline CI/CD opérationnel** : Build automatisé, scan de sécurité, déploiement progressif (dev/preprod/prod) via GitLab CI
-  **Approche GitOps** : Synchronisation automatique des déploiements Kubernetes via ArgoCD
-  **Observabilité complète** : Stack Prometheus + Grafana pour le monitoring infrastructure et applicatif
-  **Maîtrise budgétaire** : Respect strict du budget de 400€ grâce à l'optimisation

Le projet a permis de valider **l'ensemble des blocs de compétences RNCP36061**, de l'automatisation d'infrastructure (BC1) au déploiement continu (BC2) en passant par la supervision (BC3).

6.2. Difficultés rencontrées et solutions apportées



Défi organisationnel : Passage d'un projet d'équipe à un projet solo

Difficulté : Le départ successif de mes deux partenaires de formation a nécessité une réorganisation complète. Initialement prévu comme un travail collaboratif, le projet est devenu un défi individuel nécessitant la prise en charge de l'ensemble de la stack technique.

Solutions apportées :

- Réorganisation du planning et priorisation des tâches critiques
- Accompagnement hebdomadaire avec Benjamin Fiche (mentor projet) pour validation des choix techniques et déblocage des situations complexes
- Utilisation intensive des ressources communautaires (Stack Overflow, Reddit r/devops, documentation AWS)

Contrainte budgétaire : Déployer une infrastructure production-ready avec 400€

Difficulté : Le budget AWS alloué (400€) imposait une gestion rigoureuse des coûts pour l'ensemble du projet (infrastructure, stockage, transfert de données, monitoring).

Complexité technique : Courbe d'apprentissage Kubernetes et EKS

Difficulté : Configuration d'un cluster EKS production-ready avec intégration de multiples add-ons (Karpenter, EBS CSI Driver, Load Balancer Controller) sans expérience préalable en production.

Solutions apportées :

- Découverte et utilisation des **AWS EKS Blueprints for Terraform** (cf. section 6)
- Étude approfondie de la documentation officielle AWS, Kubernetes et CNCF
- Expérimentation itérative : version EC2 (échec contrôlé) → reconstruction EKS (succès)

Contraintes personnelles : Problèmes de santé impactant la disponibilité

Difficulté : Période de disponibilité réduite ralentissant l'avancement du projet.

Solutions apportées :

- Communication transparente avec l'équipe pédagogique DataScientest
- Ajustement du planning avec priorisation des livrables essentiels
- Maintien de la motivation grâce au soutien du mentor et du chef de cohorte

6.3. Principaux bénéfices tirés du projet



Au-delà de la validation des compétences RNCP, ce projet a généré des bénéfices concrets et durables :

Montée en compétences techniques :

- Maîtrise de Kubernetes en environnement de production (EKS, Helm, namespaces, PVC, Ingress)
- Expertise Terraform pour l'Infrastructure as Code (modules, state management, providers multiples)
- Compréhension approfondie de l'écosystème AWS (VPC, IAM, EKS, ECR, EBS, Route 53)
- Pratique du GitOps avec ArgoCD et de l'observabilité avec Prometheus/Grafana

Développement de soft skills :

- **Résilience** : Capacité à poursuivre un projet d'envergure malgré les départs d'équipe et les contraintes personnelles
- **Autonomie** : Gestion de bout en bout d'un projet infrastructure sans supervision permanente
- **Recherche et veille technologique** : Compétence à identifier et exploiter des ressources de qualité (EKS Blueprints, blogs AWS, communautés)
- **Gestion budgétaire** : Optimisation continue des coûts cloud sans compromis sur la qualité

Impact professionnel immédiat : Grâce aux compétences acquises durant cette formation, j'ai pu **intégrer une entreprise de robotique** pour un poste en Recherche & Développement travaillant sur Kubernetes. Par la suite, j'ai rejoint **Sanofi** en alternance, où mes compétences en infrastructure, automatisation et DevOps se sont révélées essentielles pour comprendre et contribuer aux architectures complexes du secteur pharmaceutique.

Cette expérience m'a permis de mesurer **l'envergure des infrastructures nécessaires dans le domaine de la santé**, où la fiabilité, la sécurité et la scalabilité sont critiques pour supporter des applications sauvant des vies.

6.4. Préconisations pour l'évolution du projet



Court terme (0-3 mois) : Fiabilisation et qualité

Tests automatisés end-to-end :

- Intégration de tests fonctionnels dans le pipeline CI/CD (Cypress pour le frontend, pytest pour l'API)
- Validation automatique des parcours utilisateurs critiques avant déploiement en production

Stratégie de déploiement avancée :

- Implémentation du **Canary Deployment** : déploiement progressif avec monitoring du taux d'erreur
- Configuration de **rollbacks automatiques** via ArgoCD en cas de dégradation des métriques

Enrichissement du monitoring :

- Ajout de dashboards Grafana orientés métier (nombre d'utilisateurs actifs, requêtes par indicateur)
- Configuration d'alertes Slack/email pour les incidents critiques (pods crashloop, latence > 5s)

Moyen terme (3-6 mois) : Scalabilité et performance

Service mesh (Istio ou Linkerd) :

- Amélioration de l'observabilité avec traces distribuées
- Gestion avancée du trafic (retry automatique, circuit breaker)
- Chiffrement mTLS entre microservices pour renforcement de la sécurité

Autoscaling intelligent :

- **Horizontal Pod Autoscaler (HPA)** basé sur les métriques CPU/RAM et requêtes par seconde
- **Vertical Pod Autoscaler (VPA)** pour ajustement automatique des ressources allouées
- Optimisation fine de Karpenter pour réduction supplémentaire des coûts

Cache distribué Redis :

- Mise en cache des requêtes fréquentes pour réduction de la charge PostgreSQL
- Amélioration des temps de réponse (objectif < 200ms pour 95% des requêtes)

Long terme (6-12 mois) : Architecture d'entreprise et résilience



Isolation complète des environnements :

- **VPC dédié par environnement** (dev, preprod, prod) pour isolation réseau maximale
- **Cluster EKS séparé par environnement** pour éviter toute contamination entre dev et production
- **Comptes AWS distincts** via AWS Organizations pour ségrégation administrative

Multi-cloud et haute disponibilité :

- Exploration d'une architecture **multi-cloud (AWS + GCP)** pour éviter le vendor lock-in
- Déploiement multi-régions pour disponibilité géographique (eu-west-1 + us-east-1)
- Sauvegarde cross-region des données PostgreSQL

GitOps pour l'infrastructure Terraform :

- Intégration d'**Atlantis** pour automatisation des pull requests Terraform
- Validation automatique des plans Terraform avant merge
- Traçabilité complète des changements infrastructure via Git

Contribution open-source :

- Documentation et partage des patterns développés sur GitHub
- Contribution aux projets EKS Blueprints, Karpenter ou ArgoCD
- Participation aux communautés CNCF et AWS

6.5. Ouverture : Du projet pédagogique à l'impact professionnel

Retour sur l'accroche : La technologie au service des enjeux mondiaux

Comme évoqué en introduction, **828 millions de personnes souffrent encore de la faim dans le monde**. Le projet Starling, bien qu'initialement pédagogique, démontre qu'une infrastructure DevOps moderne peut **démocratiser l'accès à des données critiques** pour la prise de décision en santé publique.

Cette problématique résonne aujourd'hui dans mon parcours professionnel chez **Sanofi**, où les infrastructures cloud supportent des applications pharmaceutiques ayant un impact direct sur la santé de millions de patients. La **fiabilité, la sécurité et la scalabilité** ne sont plus des contraintes techniques, mais des impératifs éthiques.



Perspectives professionnelles : Devenir Data Platform Engineer dans la santé

Mon objectif à long terme est de devenir **Platform Engineer** et de **construire des infrastructures capables de supporter des applications critiques dans le secteur de la santé**. Les compétences acquises durant ce projet (Kubernetes, Infrastructure as Code, observabilité, sécurité) constituent les fondations de cette ambition.

Je souhaite contribuer à des projets où la technologie devient un **levier pour sauver des vies** : plateformes de recherche médicale, infrastructures d'essais cliniques, applications de télémédecine ou systèmes de gestion hospitalière. Dans ces contextes, chaque seconde de disponibilité, chaque donnée protégée, chaque déploiement sans erreur peut avoir des conséquences directes sur des patients.

Engagement dans l'amélioration continue

Le projet Starling a confirmé mon goût pour l'**apprentissage permanent** et la **résolution de problèmes complexes**. Je suis déterminé à continuer ma montée en compétences via :

- **Certifications professionnelles** : CKA (Certified Kubernetes Administrator), AWS Solutions Architect
- **Veille technologique active** : Participation aux meetups DevOps, conférences KubeCon, blogs techniques
- **Contribution open-source** : Partage de mes apprentissages avec la communauté

Cette formation DevOps chez DataScientest a été bien plus qu'une validation de compétences techniques : elle a été le **catalyseur d'une transformation professionnelle** me permettant d'intégrer le secteur de la santé et de contribuer, à mon échelle, à des infrastructures ayant un impact positif sur la société.



ANNEXES

Annexe A : Ressources documentaires officielles

Documentation AWS

Amazon EKS (Elastic Kubernetes Service)

- Documentation officielle : <https://docs.aws.amazon.com/eks/>
- Guide de démarrage : <https://docs.aws.amazon.com/eks/latest/userguide/getting-started.html>
- Best practices : <https://aws.github.io/aws-eks-best-practices/>

Amazon VPC (Virtual Private Cloud)

- Documentation officielle : <https://docs.aws.amazon.com/vpc/>
- Guide de configuration réseau : <https://docs.aws.amazon.com/vpc/latest/userguide/>

Amazon ECR (Elastic Container Registry)

- Documentation officielle : <https://docs.aws.amazon.com/ecr/>
- Guide Docker : <https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-basics.html>

Amazon EBS (Elastic Block Store)

- Documentation officielle : <https://docs.aws.amazon.com/ebs/>
- Snapshots et sauvegardes : <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-snapshots.html>

AWS IAM (Identity and Access Management)

- Documentation officielle : <https://docs.aws.amazon.com/iam/>
- Best practices de sécurité : <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>

AWS Cost Explorer

- Documentation : <https://docs.aws.amazon.com/cost-management/latest/userguide/ce-what-is.html>
- Optimisation des coûts : <https://aws.amazon.com/aws-cost-management/>

Documentation Kubernetes et CNCF

Kubernetes

- Documentation officielle : <https://kubernetes.io/docs/>



- Concepts fondamentaux : <https://kubernetes.io/docs/concepts/>
- API Reference : <https://kubernetes.io/docs/reference/kubernetes-api/>

KubectI (CLI Kubernetes)

- Documentation : <https://kubernetes.io/docs/reference/kubectI/>
- Cheat sheet : <https://kubernetes.io/docs/reference/kubectI/cheatsheet/>

Helm (Package Manager Kubernetes)

- Documentation officielle : <https://helm.sh/docs/>
- Chart development guide : https://helm.sh/docs/chart_template_guide/
- Best practices : https://helm.sh/docs/chart_best_practices/

Karpenter

- Documentation officielle : <https://karpenter.sh/docs/>
- Getting started : <https://karpenter.sh/docs/getting-started/>
- Repository GitHub : <https://github.com/aws/karpenter>

Documentation Infrastructure as Code

Terraform

- Documentation officielle : <https://www.terraform.io/docs>
- AWS Provider : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- Module EKS officiel : <https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest>
- Module VPC officiel : <https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest>

Terraform Best Practices

- Style guide : <https://www.terraform.io/docs/language/syntax/style.html>
- State management : <https://www.terraform.io/docs/language/state/>

Documentation GitOps et CI/CD

ArgoCD

- Documentation officielle : <https://argo-cd.readthedocs.io/>
- Getting started : https://argo-cd.readthedocs.io/en/stable/getting_started/
- Best practices : https://argo-cd.readthedocs.io/en/stable/user-guide/best_practices/
- Repository GitHub : <https://github.com/argoproj/argo-cd>

GitLab CI/CD



- Documentation officielle : <https://docs.gitlab.com/ee/ci/>
- Pipeline syntax : <https://docs.gitlab.com/ee/ci/yaml/>
- Docker integration : https://docs.gitlab.com/ee/ci/docker/using_docker_build.html
- Variables et secrets : <https://docs.gitlab.com/ee/ci/variables/>

Documentation Monitoring et Observabilité

Prometheus

- Documentation officielle : <https://prometheus.io/docs/>
- Query language (PromQL) : <https://prometheus.io/docs/prometheus/latest/querying/basics/>
- Best practices : <https://prometheus.io/docs/practices/>
- Repository GitHub : <https://github.com/prometheus/prometheus>

Grafana

- Documentation officielle : <https://grafana.com/docs/>
- Dashboard creation : <https://grafana.com/docs/grafana/latest/dashboards/>
- Data sources : <https://grafana.com/docs/grafana/latest/datasources/>
- Repository GitHub : <https://github.com/grafana/grafana>

Kube-Prometheus-Stack

- Helm chart officiel : <https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>
- Documentation : <https://github.com/prometheus-operator/kube-prometheus>

Documentation Frameworks Applicatifs

Next.js (Frontend)

- Documentation officielle : <https://nextjs.org/docs>
- Deployment : <https://nextjs.org/docs/deployment>
- API Routes : <https://nextjs.org/docs/api-routes/introduction>

FastAPI (Backend)

- Documentation officielle : <https://fastapi.tiangolo.com/>
- Tutorial : <https://fastapi.tiangolo.com/tutorial/>
- Deployment : <https://fastapi.tiangolo.com/deployment/>
- Repository GitHub : <https://github.com/tyangolo/fastapi>

PostgreSQL

- Documentation officielle : <https://www.postgresql.org/docs/>
- Performance tuning : https://wiki.postgresql.org/wiki/Performance_Optimization



Annexe B : Ressources AWS EKS Blueprints

Article de blog AWS ayant inspiré l'architecture

- Bootstrapping clusters with EKS Blueprints : <https://aws.amazon.com/fr/blogs/containers/bootstrapping-clusters-with-eks-blueprints/>

Repositories GitHub EKS Blueprints

- Terraform AWS EKS Blueprints : <https://github.com/aws-ia/terraform-aws-eks-blueprints>
- EKS Blueprints Addons : <https://github.com/aws-ia/terraform-aws-eks-blueprints-addons>
- Pattern Karpenter (utilisé dans Starling) : <https://github.com/aws-ia/terraform-aws-eks-blueprints/tree/main/patterns/karpenter>

AWS Samples et exemples

- AWS Samples (exemples architectures) : <https://github.com/aws-samples>
- EKS Workshop : <https://www.eksworkshop.com/>

Annexe C : Blogs techniques et veille technologique

Blogs officiels AWS

- AWS Blog principal : <https://aws.amazon.com/blogs/>
- AWS Containers Blog : <https://aws.amazon.com/blogs/containers/>
- AWS DevOps Blog : <https://aws.amazon.com/blogs/devops/>
- AWS Compute Blog : <https://aws.amazon.com/blogs/compute/>

Blogs CNCF et Kubernetes

- CNCF Blog : <https://www.cncf.io/blog/>
- Kubernetes Blog : <https://kubernetes.io/blog/>
- Cloud Native Computing Foundation : <https://www.cncf.io/>

Blogs communautaires DevOps

- DevOps.com : <https://devops.com/>
- The New Stack : <https://thenewstack.io/>
- InfoQ DevOps : <https://www.infoq.com/devops/>

Blogs personnels influents



- Kelsey Hightower (Kubernetes) : <https://twitter.com/kelseyhightower>
- Charity Majors (Observability) : <https://charity.wtf/>
- Corey Quinn (AWS Cost Optimization) : <https://www.lastweekinaws.com/>

Annexe D : Communautés et forums

Forums techniques

- Stack Overflow (tag kubernetes) : <https://stackoverflow.com/questions/tagged/kubernetes>
- Stack Overflow (tag terraform) : <https://stackoverflow.com/questions/tagged/terraform>
- Stack Overflow (tag amazon-eks) : <https://stackoverflow.com/questions/tagged/amazon-eks>
- Server Fault : <https://serverfault.com/>

Reddit (communautés DevOps)

- r/devops : <https://www.reddit.com/r/devops/>
- r/kubernetes : <https://www.reddit.com/r/kubernetes/>
- r/aws : <https://www.reddit.com/r/aws/>
- r/terraform : <https://www.reddit.com/r/terraform/>

Slack et Discord

- Kubernetes Slack : <https://kubernetes.slack.com/>
- CNCF Slack : <https://cloud-native.slack.com/>
- ArgoCD Slack : <https://argoproj.github.io/community/join-slack>

GitHub Discussions

- Kubernetes Discussions : <https://github.com/kubernetes/kubernetes/discussions>
- Terraform Discussions : <https://discuss.hashicorp.com/c/terraform-core/>
- ArgoCD Discussions : <https://github.com/argoproj/argo-cd/discussions>

Annexe E : Outils et utilitaires DevOps

Visualisation et debugging Kubernetes

- K9s (TUI Kubernetes) : <https://k9scli.io/>
- Lens (Kubernetes IDE) : <https://k8slens.dev/>
- Kubectl/Kubens : <https://github.com/ahmetb/kubectx>

Validation et sécurité



- Trivy (scanner de vulnérabilités) : <https://github.com/aquasecurity/trivy>
- Checkov (scan IaC) : <https://www.checkov.io/>
- Kube-bench (CIS Benchmarks) : <https://github.com/aquasecurity/kube-bench>
- Polaris (best practices Kubernetes) : <https://www.fairwinds.com/polaris>

Cost Management

- Kubecost : <https://www.kubecost.com/>
- Infracost (Terraform cost estimation) : <https://www.infracost.io/>
- AWS Cost Explorer : <https://aws.amazon.com/aws-cost-management/aws-cost-explorer/>

Testing et qualité

- Terratest (tests Terraform) : <https://terratest.gruntwork.io/>
- Conftest (policy testing) : <https://www.conftest.dev/>
- Cypress (tests end-to-end) : <https://www.cypress.io/>

Annexe F : Certifications et formations

Certifications Kubernetes (CNCF)

- CKA (Certified Kubernetes Administrator) : <https://www.cncf.io/certification/cka/>
- CKAD (Certified Kubernetes Application Developer) : <https://www.cncf.io/certification/ckad/>
- CKS (Certified Kubernetes Security Specialist) : <https://www.cncf.io/certification/cks/>

Certifications AWS

- AWS Certified Solutions Architect : <https://aws.amazon.com/certification/certified-solutions-architect-associate/>
- AWS Certified DevOps Engineer : <https://aws.amazon.com/certification/certified-devops-engineer-professional/>
- AWS Certified Security Specialty : <https://aws.amazon.com/certification/certified-security-specialty/>

Formations en ligne

- DataScientest (formation DevOps) : <https://datascientest.com/formation-devops>
- A Cloud Guru : <https://acloudguru.com/>
- Linux Academy : <https://linuxacademy.com/>
- KodeKloud (Kubernetes) : <https://kodekloud.com/>



Annexe G : Organismes et institutions du secteur alimentaire

Organisation des Nations Unies pour l'alimentation et l'agriculture (FAO)

- Site officiel : <https://www.fao.org/>
- FAOSTAT (base de données) : <https://www.fao.org/faostat/>
- Rapports sur la sécurité alimentaire : <https://www.fao.org/publications/sofi/>

Banque Mondiale (données alimentaires)

- Data Catalog : <https://datacatalog.worldbank.org/>
- Open Data : <https://data.worldbank.org/>

Programme Alimentaire Mondial (PAM)

- Site officiel : <https://www.wfp.org/>
- Data et statistiques : <https://www.wfp.org/publications>

Annexe I : Ressources complémentaires

Livres recommandés

- "Kubernetes: Up and Running" - Kelsey Hightower, Brendan Burns, Joe Beda
- "Terraform: Up & Running" - Yevgeniy Brikman
- "The DevOps Handbook" - Gene Kim, Jez Humble
- "Site Reliability Engineering" - Google (gratuit en ligne : <https://sre.google/books/>)

Podcasts DevOps

- Arrested DevOps : <https://www.arresteddevops.com/>
- The Cloudcast : <https://www.thecloudcast.net/>
- Software Engineering Daily : <https://softwareengineeringdaily.com/>

Conférences majeures

- KubeCon + CloudNativeCon : <https://www.cncf.io/kubecon-cloudnativecon-events/>
- AWS re:Invent : <https://reinvent.awsevents.com/>
- HashiConf : <https://hashiconf.com/>

Toutes les URL ont été vérifiées et sont accessibles à la date de rédaction de ce document (février 2026). Certaines ressources peuvent évoluer ou être déplacées avec le temps.